TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT
VIỆN CÔNG NGHỆ SỐ

BÁO CÁO TỔNG KẾT

ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN
NĂM HỌC 2025-2026

Xây dựng hệ thống hỗ trợ phân tích và phát hiện trong các dữ liệu gửi đến dựa trên đặc trưng tĩnh và kỹ thuật học máy

Sinh viên/Nhóm Sinh viên thực hiện:
Ngô Hồng Phúc và Nguyễn Hoàng Sơn
Giảng viên hướng dẫn:
ThS. Trần Bá Minh Sơn

Thành phố Hồ Chí Minh, ngày 08 tháng 06 năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT

VIỆN CÔNG NGHỆ SỐ

BÁO CÁO TỔNG KẾT ĐỀ TÀI NGHIÊN CỨU KHOA HỌC CỦA SINH VIÊN

NĂM HỌC 2025 – 2026

XÂY DỰNG HỆ THỐNG HỖ TRỢ PHÂN TÍCH VÀ PHÁT HIỆN MÃ ĐỘC DỰA TRÊN ĐẶC TRƯNG TĨNH VÀ KỸ THUẬT HỌC MÁY

STT	Họ và tên SV	Giới tính	Dân tộc	Lớp, Khoa	Năm thứ / Số năm ĐT	Ngành học	Ghi chú
1	Ngô Hồng Phúc	Nam	Kinh	D24CNTT05, Viện Công nghệ số	2 / 4.5	CNTT	Chính
2	Nguyễn Hoàng Sơn	Nam	Kinh	D24CNTT05, Viện Công nghệ số	2 / 4.5	CNTT	
3							
4							
5							

Giảng viên hướng dẫn: ThS.Trần Bá Minh Sơn

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

THÔNG TIN KẾT QUẢ NGHIÊN CỨU CỦA ĐỀ TÀI

1. Thông tin chung:

– Tên đề tài: *Xây dựng hệ thống hỗ trợ phân tích và phát hiện mã độc dựa trên đặc trưng tĩnh và kỹ thuật học máy trên nền tảng website*

– Sinh viên / nhóm sinh viên thực hiện:

STT	Họ và tên	MSSV	Lớp	Khoa	Năm thứ / Số năm ĐT
1	Ngô Hồng Phúc	2424802010413	D24CNTT05	Viện Công Nghệ Số	2 / 4.5
2	Nguyễn Hoàng Sơn	2424802010430	D24CNTT05	Viện Công Nghệ Số	2 / 4.5
3					

– Người hướng dẫn: ThS.Trần Bá Minh Sơn

2. Mục tiêu đề tài:

Nghiên cứu xây dựng hệ thống tự động phát hiện mã độc dạng file PE (Portable Executable) dựa trên phân tích đặc trưng tĩnh kết hợp học máy, triển khai dưới dạng ứng dụng web tương tác có khả năng giải thích quyết định phân loại và tối ưu hóa ngưỡng phân loại.

3. Tính mới và sáng tạo:

Đề tài có ba đóng góp chính chưa được kết hợp đầy đủ trong các nghiên cứu trước: (1) áp dụng Youden's J Statistic để tối ưu hóa ngưỡng phân loại, giảm 43% tỷ lệ cảnh báo nhầm (FPR từ 3,59% xuống 2,05%); (2) xây dựng Evidence Aggregation Framework gồm năm chỉ số bằng chứng liên tục kết hợp với SHAP per - sample để giải thích quyết định; (3) thiết kế Import Pattern Engine phân loại API theo tám danh mục MITRE ATT&CK với coverage 45,1% trên mẫu mã độc.

4. Kết quả nghiên cứu:

XGBoost đạt F1 = 0,9682 và AUC = 0,9938 trên tập kiểm thử độc lập 200.000 mẫu. Holdout Stability Test (std F1 = 0,00023 qua 8 random seed) xác nhận mô hình ổn định, không phụ thuộc cách chia dữ liệu. Hệ thống web Flask tích hợp đầy đủ pipeline với chat log real - time và giải thích ngôn ngữ tự nhiên qua hàm mock_ai_interpretation().

5. Đóng góp về kinh tế – xã hội và khả năng áp dụng:

Hệ thống có thể được ứng dụng như công cụ hỗ trợ phân tích bảo mật tại các tổ chức vừa và nhỏ, phục vụ đào tạo sinh viên ngành an toàn thông tin, và làm nền tảng nghiên cứu mở rộng tích hợp phân tích động trong tương lai.

6. Công bố khoa học:

Đề tài chưa có công bố khoa học tại thời điểm nộp báo cáo. Kết quả nghiên cứu dự kiến được trình bày tại hội thảo khoa học cấp trường năm học 2025–2026.

Thành phố Hồ Chí Minh, ngày tháng năm 2026

SINH VIÊN

CHỊU TRÁCH NHIỆM CHÍNH

Phúc

Ngô Hồng Phúc

Nhận xét của người hướng dẫn về những đóng góp khoa học của sinh viên:

Thành phố Hồ Chí Minh, ngày 08 tháng 06 năm 2026

TRƯỞNG ĐƠN VỊ	**GIẢNG VIÊN HƯỚNG DẪN**
(ký, họ và tên)	*(ký, họ và tên)*



ỦY BAN NHÂN DÂN
THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

THÔNG TIN VỀ SINH VIÊN CHỊU TRÁCH NHIỆM CHÍNH THỰC HIỆN ĐỀ TÀI

I. SƠ LƯỢC VỀ SINH VIÊN:

Họ và tên: Ngô Hồng Phúc

Sinh ngày: 24 tháng 07 năm 2006

Nơi sinh: Bình Dương

Lớp: D24CNTT05 Khóa: 2024-2029

Khoa/viện: Viện Công Nghệ Số

Địa chỉ liên hệ: 23, DX068, khu 6, phường Chánh Hiệp, thành phố Hồ Chí Minh

Điện thoại: 0777268757 Email: 2424802010413@student.tdmu.edu.vn

II. QUÁ TRÌNH HỌC TẬP (kê khai thành tích của sinh viên từ năm thứ 1 đến năm đang học):

**** Năm thứ 1:***

Ngành học: Công nghệ thông tin Khoa/viện: Viện Công Nghệ Số

Kết quả xếp loại học tập: 7.47

Sơ lược thành tích: HK1: 8.18, HK2:7.47, HK3: 6.96

**** Năm thứ 2:***

Ngành học: Công nghệ thông tin Khoa/viện: Viện Công Nghệ Số

Kết quả xếp loại học tập: 7.42

Sơ lược thành tích: HK1: 7.59, HK2: 7.1 HK3: Chưa xong

Ngày 08 tháng 06 năm 2026

XÁC NHẬN CỦA ĐƠN VỊ **XÁC NHẬN CỦA SINH VIÊN**

(ký, họ và tên) *(ký, họ và tên)*

Phúc

Ngô Hồng Phúc

LỜI CẢM ƠN

Nhóm tác giả xin gửi lời cảm ơn sâu sắc đến Giảng viên hướng dẫn đã tận tình hỗ trợ, định hướng nghiên cứu và góp ý trong suốt quá trình thực hiện đề tài. Sự hướng dẫn khoa học nghiêm túc và chuyên sâu của Thầy/Cô là nền tảng quan trọng để đề tài được hoàn thành đúng tiến độ và đảm bảo chất lượng học thuật.

Nhóm tác giả cũng xin cảm ơn Ban Giám hiệu Trường Đại học Thủ Dầu Một, Ban Chủ nhiệm Khoa Công nghệ Thông tin, và các Thầy/Cô trong Khoa đã tạo điều kiện học thuật và cơ sở vật chất thuận lợi cho việc nghiên cứu. Sự hỗ trợ từ cộng đồng mã nguồn mở - đặc biệt nhóm tác giả EMBER 2018, thư viện SHAP, LIEF, XGBoost, LightGBM - đã đóng góp trực tiếp vào khả năng thực hiện nghiên cứu này.

Mặc dù đã nỗ lực tối đa, đề tài không tránh khỏi những thiếu sót. Nhóm tác giả kính mong nhận được ý kiến đóng góp từ Hội đồng để tiếp tục hoàn thiện và phát triển trong tương lai.

LỜI CAM ĐOAN

Chúng tôi cam đoan đây là công trình nghiên cứu do chúng tôi thực hiện. Các số liệu, kết quả thực nghiệm nêu trong báo cáo là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác.

Toàn bộ kết quả thực nghiệm trong báo cáo này được thực hiện trực tiếp bởi nhóm tác giả trên bộ dữ liệu EMBER 2018 thực tế (600.000 mẫu có nhãn). Quy trình thực nghiệm được thiết kế đảm bảo không có data leakage, với Holdout Stability Test xác nhận kết quả không phụ thuộc cách chia dữ liệu. Mã nguồn, file model (.pkl), và kết quả đầy đủ được lưu trữ và có thể tái lập.

TÓM TẮT

Nghiên cứu này trình bày quá trình xây dựng hệ thống hỗ trợ phát hiện mã độc dựa trên phân tích đặc trưng tĩnh của file Portable Executable (PE) kết hợp với các kỹ thuật học máy, được triển khai dưới dạng ứng dụng web tương tác. Hệ thống sử dụng bộ dữ liệu EMBER 2018 [1] với 600.000 mẫu có nhãn, thực hiện trích xuất 2.381 đặc trưng tĩnh bao gồm histogram byte, entropy cục bộ, thông tin header PE, thông tin section và bảng import thông qua thư viện LIEF phiên bản 0.17.6.

Ba mô hình học máy được huấn luyện và đánh giá: XGBoost [3], LightGBM [4] và Random Forest [5]. XGBoost đạt kết quả tốt nhất với F1 = 0,9682 và AUC = 0,9938 trên tập kiểm thử độc lập 200.000 mẫu. Holdout Stability Test với 8 random seed cho độ lệch chuẩn F1 = 0,00023, xác nhận mô hình không phụ thuộc cách phân chia dữ liệu.

Nghiên cứu đề xuất ba đóng góp chính: (1) áp dụng Youden's J Statistic [6] để tối ưu hóa ngưỡng phân loại, giảm 43% tỷ lệ cảnh báo nhầm so với ngưỡng mặc định 0,5; (2) xây dựng Evidence Aggregation Framework kết hợp SHAP [2] để giải thích quyết định per - sample thông qua năm chỉ số bằng chứng liên tục; (3) thiết kế Import Pattern Engine phân loại API theo tám danh mục MITRE ATT&CK [8] đạt coverage 45,1% trên tập mẫu mã độc.

Hệ thống web Flask tích hợp đầy đủ pipeline phân tích, hiển thị kết quả qua chat log real - time với giải thích ngôn ngữ tự nhiên thông qua hàm mock_ai_interpretation(). Nghiên cứu cũng xác định rõ các giới hạn của phương pháp phân tích tĩnh đối với stager/dropper và fileless malware.

Từ khóa: phát hiện mã độc, đặc trưng tĩnh, học máy, XGBoost, SHAP, EMBER 2018, PE file, Youden's J, MITRE ATT&CK.

MỤC LỤC

LỜI CẢM ƠN i
LỜI CAM ĐOAN ii
TÓM TẮT iii
DANH MỤC BẢNG VÀ HÌNH VẼ vi
DANH MỤC CÁC CHỮ VIẾT TẮT vii
CHƯƠNG 1: GIỚI THIỆU 1
1.1. Tính cấp thiết của đề tài 1
1.2. Mục tiêu nghiên cứu 1
1.3. Phạm vi nghiên cứu 2
1.4. Cấu trúc báo cáo 2
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÀI LIỆU 3
2.1. Định dạng file PE và đặc trưng tĩnh 3
2.2. Các thuật toán học máy áp dụng 3
2.2.1. XGBoost 3
2.2.2. LightGBM 4
2.2.3. Random Forest 4
2.3. Youden's J Statistic 4
2.4. SHAP - Giải thích mô hình học máy 5
2.5. Tổng quan các nghiên cứu liên quan 5
CHƯƠNG 3: DỮ LIỆU VÀ PHƯƠNG PHÁP NGHIÊN CỨU 7
3.1. Bộ dữ liệu EMBER 2018 7
3.2. Phương pháp trích xuất đặc trưng 7
3.3. Thiết kế quy trình thực nghiệm 8
CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG 10
4.1. Kiến trúc tổng thể 10
4.2. Training Pipeline - 7 bước xử lý 11
Bước 1 - Vectorize (Step 1) 12
Bước 2 - Split & Scale (Step 2) 12
Bước 3 - Train Models (Step 3) 13
Bước 4 - Evaluate (Step 4) 13
Bước 5 - SHAP Analysis (Step 5) 14
Bước 6 - Evidence Framework (Step 6) 14
Bước 7 - Import Pattern (Step 7) 15
Kết luận thực tế: 15

4.3. Evidence Aggregation Framework 15
4.4. Import Pattern Engine 16
4.5. Ứng dụng web Flask 17
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN 21
5.1. Kết quả huấn luyện mô hình 21
5.2. Đánh giá trên tập kiểm thử độc lập 21
5.3. Phân tích SHAP và độ ổn định đặc trưng 23
5.4. Kết quả Robustness Test 26
5.5. Kết quả Import Pattern Coverage 27
5.6. Hạn chế của hệ thống 28
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 30
6.1. Kết luận 30
6.2. Hướng phát triển 30
TÀI LIỆU THAM KHẢO 32
PHỤ LỤC 33
Phụ lục A - Cấu trúc thư mục dự án 33
Phụ lục B - Phiên bản thư viện và công cụ 33
Phụ lục C - Cấu trúc JSON output của pipeline 33
Phụ lục D - Kết quả thực nghiệm đầy đủ 34

DANH MỤC BẢNG VÀ HÌNH VẼ

DANH MỤC BẢNG

Bảng 1. So sánh nghiên cứu này với các công trình liên quan 6
Bảng 2. Các nhóm đặc trưng trích xuất và số chiều tương ứng 8
Bảng 3. Import Pattern Engine - 8 danh mục và thông tin MITRE ATT&CK tương ứng 17
Bảng 4. Kết quả Cross - Validation 5 - Fold và thời gian thực thi 21
Bảng 5. Kết quả tập kiểm thử - ngưỡng mặc định và Youden's J tối ưu 22
Bảng 6. SHAP Stability Score - Top 10 đặc trưng quan trọng nhất 23
Bảng 7. Kết quả Robustness Test - F1 theo mức nhiễu Gaussian 26
Bảng 8. Phiên bản thư viện và công cụ sử dụng trong nghiên cứu 33

DANH MỤC HÌNH VẼ

Hình 1. Kiến trúc tổng thể hệ thống phát hiện mã độc 10
Hình 2. Training Pipeline - sơ đồ luồng xử lý và quan hệ giữa 7 bước 11
Hình 3. Evidence Vector Radar Chart - so sánh malware (cam) và benign (xanh) 16
Hình 4. Giao diện chờ — vùng upload file PE và nhật ký phân tích trống 18
Hình 5. BENIGN 44,7% — phát hiện process injection API nhưng mô hình không đủ bằng chứng kết luận malware 19
Hình 6. MALWARE 92,7% — Packing Evidence tăng lên 43%, SHAP TLS Table là tín hiệu chính (+0,441) 20
Hình 7. Feature Importance - so sánh tầm quan trọng đặc trưng theo ba mô hình 21
Hình 8. ROC Curve - ba mô hình trên tập kiểm thử 200.000 mẫu 22
Hình 9. Threshold Analysis - Youden's J Optimization cho XGBoost 23
Hình 10. SHAP Summary Plot - XGBoost, Top 10 đặc trưng (màu đỏ = giá trị cao, màu xanh = giá trị thấp) 24
Hình 11. SHAP Waterfall Plot - giải thích quyết định cho mẫu malware cụ thể 25
Hình 12. SHAP Stability Chart - Stability Score của 10 đặc trưng quan trọng nhất 25
Hình 13. Robustness Curve - F1 theo mức nhiễu Gaussian σ (XGBoost giảm ít nhất) 27
Hình 14. Import Pattern Coverage - tỷ lệ phát hiện theo từng danh mục (malware vs benign) 28

Hình D 1. Overlap đặc trưng trong top - 20 SHAP giữa ba mô hình 34
Hình D 2. So sánh F1 / FPR / FNR giữa ML - only, Evidence - only, và Final Score 34
Hình D 3. So sánh CMS (Composite Malware Score) với các phương pháp đơn lẻ 35

DANH MỤC CÁC CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh đầy đủ	Nghĩa tiếng Việt
AUC	Area Under the Curve	Diện tích dưới đường cong ROC
API	Application Programming Interface	Giao diện lập trình ứng dụng
CMS	Composite Malware Score	Điểm mã độc tổng hợp
COFF	Common Object File Format	Định dạng file đối tượng chung
DLL	Dynamic - Link Library	Thư viện liên kết động
EFB	Exclusive Feature Bundling	Gộp đặc trưng loại trừ nhau (LightGBM)
EMBER	Elastic Malware Benchmark for Empowering Researchers	Bộ dữ liệu chuẩn cho nghiên cứu phát hiện mã độc
EMU	English Metric Unit	Đơn vị đo lường trong tài liệu Office (1 inch = 914400 EMU)
FNR	False Negative Rate	Tỷ lệ bỏ sót mã độc
FPR	False Positive Rate	Tỷ lệ cảnh báo nhầm
GOSS	Gradient - based One - Side Sampling	Lấy mẫu một phía dựa trên gradient
JSON	JavaScript Object Notation	Định dạng dữ liệu dạng văn bản nhẹ
LIEF	Library to Instrument Executable Formats	Thư viện phân tích định dạng file thực thi
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MITRE	MITRE Corporation	Tổ chức phi lợi nhuận quản lý khung ATT&CK
ML	Machine Learning	Học máy
PE	Portable Executable	Định dạng file thực thi trên Windows
ROC	Receiver Operating Characteristic	Đường cong đặc tính hoạt động của bộ phân loại
SHAP	SHapley Additive exPlanations	Phương pháp giải thích mô hình dựa trên giá trị Shapley
SSE	Server - Sent Events	Sự kiện đẩy từ máy chủ theo thời gian thực
TLS	Thread Local Storage	Vùng lưu trữ cục bộ luồng trong PE
TN	True Negative	Dự đoán đúng - benign là benign
TP	True Positive	Dự đoán đúng - malware là malware
XAI	Explainable Artificial Intelligence	Trí tuệ nhân tạo có khả năng giải thích
XGBoost	eXtreme Gradient Boosting	Thuật toán gradient boosting hiệu năng cao

CHƯƠNG 1: GIỚI THIỆU

1.1. Tính cấp thiết của đề tài

Mã độc (malware) là mối đe dọa ngày càng nghiêm trọng trong lĩnh vực an ninh mạng, gây thiệt hại hàng tỷ đô la mỗi năm cho cá nhân, tổ chức và chính phủ trên toàn thế giới. Theo báo cáo của các tổ chức bảo mật uy tín, số lượng mẫu mã độc mới xuất hiện mỗi ngày lên đến hàng triệu, vượt xa khả năng xử lý thủ công của chuyên gia phân tích bảo mật. Các phương pháp phát hiện truyền thống dựa trên chữ ký (signature - based detection) ngày càng kém hiệu quả trước biến thể mã độc mới, kỹ thuật obfuscation tinh vi và các cuộc tấn công zero - day.

Trong bối cảnh đó, nghiên cứu áp dụng học máy vào phân tích đặc trưng tĩnh của file PE (Portable Executable) - định dạng file thực thi chuẩn trên nền tảng Windows - là hướng tiếp cận được cộng đồng học thuật và công nghiệp quan tâm mạnh mẽ từ sau công bố benchmark EMBER 2018 của Anderson và Roth [1]. Phân tích tĩnh không cần thực thi file trong môi trường sandbox, do đó nhanh hơn và an toàn hơn so với phân tích động, đồng thời có thể phát hiện nhiều loại mã độc chưa từng xuất hiện trong cơ sở dữ liệu chữ ký.

Tuy nhiên, hầu hết các nghiên cứu hiện có chỉ tập trung tối ưu hóa chỉ số F1 hay AUC mà không giải thích được lý do mô hình đưa ra quyết định. Điều này tạo ra các hệ thống "hộp đen" khó được chấp nhận trong môi trường thực tế, nơi chuyên gia bảo mật cần hiểu tại sao một file cụ thể bị phán là mã độc. Thêm vào đó, ngưỡng phân loại mặc định 0,5 không được tối ưu hóa cho tương quan giữa chi phí false positive và false negative trong bài toán bảo mật. Đề tài này giải quyết đồng thời hai khoảng trống đó.

1.2. Mục tiêu nghiên cứu

Nghiên cứu đặt ra các mục tiêu cụ thể sau:

– Xây dựng pipeline trích xuất đặc trưng tĩnh từ file PE bằng thư viện LIEF và EMBER, đảm bảo tính tái lập và không có data leakage.

- Huấn luyện và so sánh ba mô hình học máy - XGBoost, LightGBM và Random Forest - trên bộ dữ liệu EMBER 2018 theo quy trình Cross - Validation nghiêm túc.
- Đề xuất và kiểm chứng phương pháp tối ưu hóa ngưỡng phân loại dựa trên Youden's J Statistic, giảm tỷ lệ cảnh báo nhầm trong môi trường thực tế.
- Xây dựng Evidence Aggregation Framework kết hợp SHAP để cung cấp giải thích per - sample có ngữ nghĩa bảo mật cho mỗi quyết định phân loại.
- Thiết kế Import Pattern Engine phân loại API đáng ngờ theo danh mục MITRE ATT&CK nhằm tăng cường khả năng phát hiện và giải thích của hệ thống.
- Triển khai ứng dụng web Flask tích hợp đầy đủ pipeline phân tích với giao diện chat log theo thời gian thực.

1.3. Phạm vi nghiên cứu

Nghiên cứu tập trung vào phân tích đặc trưng tĩnh của file PE (định dạng .exe và .dll) trên nền tảng Windows, sử dụng bộ dữ liệu EMBER 2018 làm benchmark huấn luyện và đánh giá. Hệ thống không thực hiện phân tích động (dynamic analysis) hay thực thi file trong sandbox. Các loại mã độc phi PE (macro, script, firmware) nằm ngoài phạm vi nghiên cứu này.

1.4. Cấu trúc báo cáo

Báo cáo được tổ chức thành sáu chương. Chương 1 giới thiệu đề tài và mục tiêu nghiên cứu. Chương 2 trình bày cơ sở lý thuyết và tổng quan tài liệu liên quan. Chương 3 mô tả bộ dữ liệu và phương pháp nghiên cứu. Chương 4 trình bày thiết kế và cài đặt hệ thống. Chương 5 trình bày kết quả thực nghiệm và thảo luận. Chương 6 đưa ra kết luận và hướng phát triển.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN TÀI LIỆU

2.1. Định dạng file PE và đặc trưng tĩnh

File PE (Portable Executable) là định dạng file thực thi tiêu chuẩn trên hệ điều hành Windows, bao gồm các thành phần cấu trúc cơ bản sau: DOS Header (chứa chữ ký MZ 0x4D5A và con trỏ đến PE Header), COFF File Header (lưu machine type, số section, timestamp biên dịch, và COFF Characteristics), Optional Header (chứa entry point, subsystem, phiên bản, kích thước stack/heap, và bảng data directory), Section Table (mô tả các vùng dữ liệu như .text, .data, .rdata, .rsrc với thông tin kích thước và quyền truy cập), và Data Directories (con trỏ đến các cấu trúc đặc biệt: Import Table, Export Table, TLS Table, Load Configuration Directory, Delay Import Descriptor).

Từ cấu trúc PE, các nghiên cứu đã xác định nhiều nhóm đặc trưng tĩnh có giá trị phân biệt giữa mã độc và phần mềm hợp lệ. Kolter và Maloof [10] là những người tiên phong sử dụng byte n - gram từ raw byte stream (2006). Saxe và Berlin [9] đề xuất sử dụng byte histogram hai chiều với mạng neural sâu (2015). Anderson và Roth [1] tổng hợp nhiều nhóm đặc trưng thành vector 2.381 chiều (phiên bản LIEF 0.17.6) bao phủ toàn bộ các thành phần cấu trúc PE, tạo nên bộ đặc trưng phong phú nhất trong các nghiên cứu static analysis hiện có.

2.2. Các thuật toán học máy áp dụng

2.2.1. XGBoost

XGBoost (eXtreme Gradient Boosting) [3] là framework gradient boosting hiệu quả cao, sử dụng regularization để kiểm soát độ phức tạp mô hình. Hàm mục tiêu tối ưu hóa bao gồm hàm mất mát L và số hạng điều hòa Ω:

$$L(\varphi) = \Sigma_i\, l(\hat{y}_i, y_i) + \Sigma_k\, \Omega(f_k), \quad \text{với} \quad \Omega(f) = \gamma T + (1/2)\lambda ||w||^2$$

trong đó T là số lượng lá, w là vector trọng số lá, γ và λ là siêu tham số điều hòa. XGBoost sử dụng xấp xỉ bậc hai của hàm mất mát (thông tin gradient bậc nhất g_i và bậc hai h_i) để tối ưu hóa hiệu quả hơn so với gradient boosting truyền thống. Ngoài ra,

XGBoost hỗ trợ xử lý giá trị thiếu, column subsampling và tính toán song song theo cột.

2.2.2. LightGBM

LightGBM [4] sử dụng hai kỹ thuật cốt lõi: Gradient - based One - Side Sampling (GOSS) - loại bỏ các mẫu có gradient nhỏ trong khi giữ lại mẫu gradient lớn; và Exclusive Feature Bundling (EFB) - gộp các đặc trưng thưa loại trừ nhau thành bundle, giảm số đặc trưng hiệu dụng. Chiến lược tăng trưởng cây theo chiều lá (leaf - wise) thay vì theo chiều sâu (depth - wise) giúp LightGBM hội tụ nhanh hơn trên dữ liệu lớn.

2.2.3. Random Forest

Random Forest [5] là phương pháp ensemble sử dụng bootstrap aggregating (bagging) kết hợp với chọn ngẫu nhiên tập con đặc trưng tại mỗi nút phân chia. Xác suất phân loại cuối bằng trung bình xác suất của T cây:

$$P(y = 1 \mid x) = (1/T)\ \Sigma_t\ P_t(y = 1 \mid x)$$

Sự đa dạng giữa các cây được đảm bảo bởi bootstrap sampling và random feature selection, giúp giảm phương sai tổng thể. Tuy nhiên, Random Forest có thời gian inference chậm hơn đáng kể so với gradient boosting do cần duyệt qua nhiều cây lớn.

2.3. Youden's J Statistic

Youden's J Statistic [6] là phương pháp tối ưu hóa ngưỡng phân loại dựa trên đường cong ROC (Receiver Operating Characteristic), tìm ngưỡng τ* tối đa hóa chỉ số J:

$$J = max_\tau\ [\ TPR(\tau)\ -\ FPR(\tau)\ ] = max_\tau\ [\ Sensitivity(\tau) + Specificity(\tau)\ -\ 1\ ]$$

Ngưỡng τ* tối ưu cân bằng giữa Sensitivity (khả năng bắt đúng mã độc) và Specificity (tránh cảnh báo nhầm), thay vì nghiêng về một chiều như ngưỡng mặc định 0,5. Trong bài toán phát hiện mã độc, cả false positive (file hợp lệ bị phán là mã độc) và false negative (mã độc bị bỏ sót) đều có chi phí thực tế quan trọng, nên Youden's J là lựa chọn phù hợp hơn so với tối ưu hóa F1 hay accuracy đơn thuần.

2.4. SHAP - Giải thích mô hình học máy

SHAP (SHapley Additive exPlanations) [2] dựa trên lý thuyết Shapley value từ game theory, phân bổ công bằng đóng góp của từng đặc trưng vào dự đoán của mô hình cho một mẫu cụ thể. Với mô hình f(x), SHAP value của đặc trưng i cho mẫu x được xác định duy nhất bởi bốn tiên đề: hiệu quả (tổng SHAP bằng sự khác biệt dự đoán so với baseline), đối xứng, dummy, và tính tuyến tính.

TreeExplainer - thuật toán SHAP chuyên dụng cho mô hình cây - tính giá trị SHAP chính xác (không xấp xỉ) trong thời gian $O(TLD^2)$, với T là số cây, L là số lá tối đa, D là chiều sâu tối đa. SHAP Stability Score được định nghĩa trong nghiên cứu này là 1 - CV(|SHAP|), đo độ ổn định của SHAP value qua nhiều subset ngẫu nhiên: đặc trưng có Stability Score cao là đặc trưng discriminative ổn định, không phụ thuộc phân phối cụ thể của dataset huấn luyện.

2.5. Tổng quan các nghiên cứu liên quan

Anderson và Roth [1] công bố bộ dữ liệu EMBER 2018 và mô hình LightGBM baseline đạt AUC = 0,9911, thiết lập benchmark chuẩn được sử dụng rộng rãi trong cộng đồng nghiên cứu. Nghiên cứu gốc không thực hiện threshold optimization và không cung cấp khả năng giải thích quyết định - đây là hai khoảng trống mà đề tài này giải quyết.

Một nghiên cứu IEEE (2024) áp dụng LightGBM kết hợp SHAP trên EMBER 2018, đạt accuracy 91,16% và AUC 0,9744. Nghiên cứu này chứng minh SHAP có khả năng highlight đặc trưng nhị phân quan trọng, nhưng chưa thực hiện threshold optimization hay Import Pattern Analysis. FutureComputing4AI [7] công bố EMBER 2024 với 3,2 triệu mẫu và challenge set 6.315 mẫu evasive malware - là cơ sở thực tế cho Robustness Test trong nghiên cứu này.

Nghiên cứu	Dataset	Phương pháp	AUC / F1	Điểm khác biệt
Anderson & Roth [1] (2018)	EMBER 2018	LightGBM	AUC 0,9911	Không có XAI, ngưỡng mặc định
IEEE (2024)	EMBER 2018	LightGBM + SHAP	Acc 91,16%	Không threshold opt., không Import Pattern
Nghiên cứu này	EMBER 2018	XGBoost + SHAP + Youden's J	AUC 0,9938; F1 0,9682	Evidence Framework + Import Pattern + Web

Bảng 1. So sánh nghiên cứu này với các công trình liên quan

- **Khoảng trống nghiên cứu**: Qua khảo sát các nghiên cứu tiêu biểu giai đoạn 2018–2026, có thể thấy các công trình hiện nay chủ yếu tập trung vào từng khía cạnh riêng lẻ. Anderson và Roth (2018) với bộ dữ liệu EMBER đã đặt nền tảng cho phát hiện mã độc dựa trên đặc trưng PE tĩnh, tuy nhiên chưa đề cập đến tối ưu ngưỡng phân loại, cơ chế giải thích kết quả hay ánh xạ theo khung tri thức MITRE ATT&CK. Các nghiên cứu cải tiến mô hình trên EMBER như Gao et al. (2022) và Barut et al. (2023) chủ yếu tập trung nâng cao hiệu năng phát hiện thông qua tối ưu thuật toán và đặc trưng, nhưng chưa tích hợp khả năng giải thích theo ngữ nghĩa an ninh mạng. Trong khi đó, các nghiên cứu về Explainable AI đã áp dụng các kỹ thuật như SHAP nhằm giải thích dự đoán của mô hình, tuy nhiên chủ yếu dừng ở việc xác định mức độ đóng góp của đặc trưng, chưa kết hợp Youden's J để tối ưu ngưỡng phân loại và cũng chưa xây dựng cơ chế ánh xạ Import API sang MITRE ATT&CK. Bên cạnh đó, các nghiên cứu ứng dụng MITRE ATT&CK chủ yếu dựa trên dynamic analysis hoặc dữ liệu hành vi trong quá trình thực thi, thay vì khai thác trực tiếp Import Table của tệp PE trong phân tích tĩnh. Do đó, trong phạm vi khảo sát các tài liệu giai đoạn 2018–2026, chúng tôi chưa tìm thấy công trình nào đồng thời tích hợp: (i) Youden's J threshold optimization, (ii) Evidence Aggregation Framework kết hợp SHAP để giải thích ở mức từng mẫu (*per-sample*), và (iii) Import Pattern Engine ánh xạ API import tĩnh sang danh mục MITRE ATT&CK trong một hệ thống thống nhất có khả năng triển khai trên nền tảng web.

CHƯƠNG 3: DỮ LIỆU VÀ PHƯƠNG PHÁP NGHIÊN CỨU

3.1. Bộ dữ liệu EMBER 2018

EMBER 2018 (Elastic Malware Benchmark for Empowering Researchers) [1] là benchmark chuẩn được thiết kế tương tự ImageNet cho bài toán phân loại mã độc file PE, bao gồm 1.000.000 mẫu file PE Windows. Phân phối nhãn: 300.000 mẫu benign có nhãn, 300.000 mẫu malware có nhãn, và 400.000 mẫu không có nhãn (unlabeled, label = - 1).

Trong nghiên cứu này, 400.000 mẫu unlabeled được loại bỏ trước khi huấn luyện, để lại 600.000 mẫu có nhãn cân bằng hoàn toàn (300.000 benign, 300.000 malware). Tập huấn luyện hiệu dụng là 510.000 mẫu (85%) và tập validation là 90.000 mẫu (15%). Tập kiểm thử độc lập có 200.000 mẫu (100.000 benign, 100.000 malware), không được sử dụng trong bất kỳ bước huấn luyện hay tuning nào.

Đặc điểm nổi bật của EMBER 2018: các đặc trưng được trích xuất sẵn từ file PE thực tế qua LIEF, không chứa raw bytes nguy hiểm; dữ liệu được chia theo thời gian (time - based split) với tập test chứa mẫu xuất hiện muộn hơn tập train, phản ánh thực tế triển khai; và được sử dụng rộng rãi trong hơn 100 công trình nghiên cứu từ năm 2018 đến nay.

3.2. Phương pháp trích xuất đặc trưng

Quá trình trích xuất đặc trưng sử dụng EMBER PEFeatureExtractor với feature_version=2, kết hợp thư viện LIEF phiên bản 0.17.6. Tổng số đặc trưng sau trích xuất là 2.381 chiều (thay vì 2.351 chiều của EMBER gốc do sự khác biệt phiên bản LIEF - phần chênh lệch 30 chiều thuộc nhóm DataDirectories). Các đặc trưng không được chuẩn hóa trong bước này; StandardScaler được áp dụng riêng biệt ở bước tiếp theo.

Nhóm đặc trưng	Số chiều	Mô tả
Byte Histogram	256	Tần suất xuất hiện của 256 giá trị byte 0x00–0xFF
Byte Entropy Histogram	256	Entropy Shannon theo cửa sổ trượt 2.048 byte
String Extraction	104	Chuỗi printable, URL, đường dẫn registry, chuỗi MZ header
General File Info	10	Kích thước file, có debug/TLS/exports, v.v.
Header File Info	62	Timestamp, machine type, subsystem, DLL characteristics
Section Info	255	Entropy, kích thước, quyền truy cập (RWX) từng section
Imports Info	1.280	FeatureHasher trên tên DLL và tên hàm import
Exports Info	128	FeatureHasher trên tên hàm export
Data Directories	30	Kích thước TLS Table, Load Config, Delay Import, v.v.
Tổng cộng	2.381	

Bảng 2. Các nhóm đặc trưng trích xuất và số chiều tương ứng

Trong quá trình triển khai thực tế, đã phát hiện và khắc phục hai vấn đề tương thích thư viện: (1) hàm numpy np.int đã bị xóa trong NumPy 2.0 - được sửa thành np.int64; (2) FeatureHasher trong SectionInfo.process_raw_features yêu cầu danh sách danh sách chuỗi thay vì danh sách chuỗi đơn - được sửa bằng cách thêm lớp bọc ngoài. Hai bản vá này được áp dụng trực tiếp vào file features.py của thư viện ember.

3.3. Thiết kế quy trình thực nghiệm

Quy trình thực nghiệm được thiết kế tuân thủ các nguyên tắc tránh data leakage và đảm bảo tính tái lập:

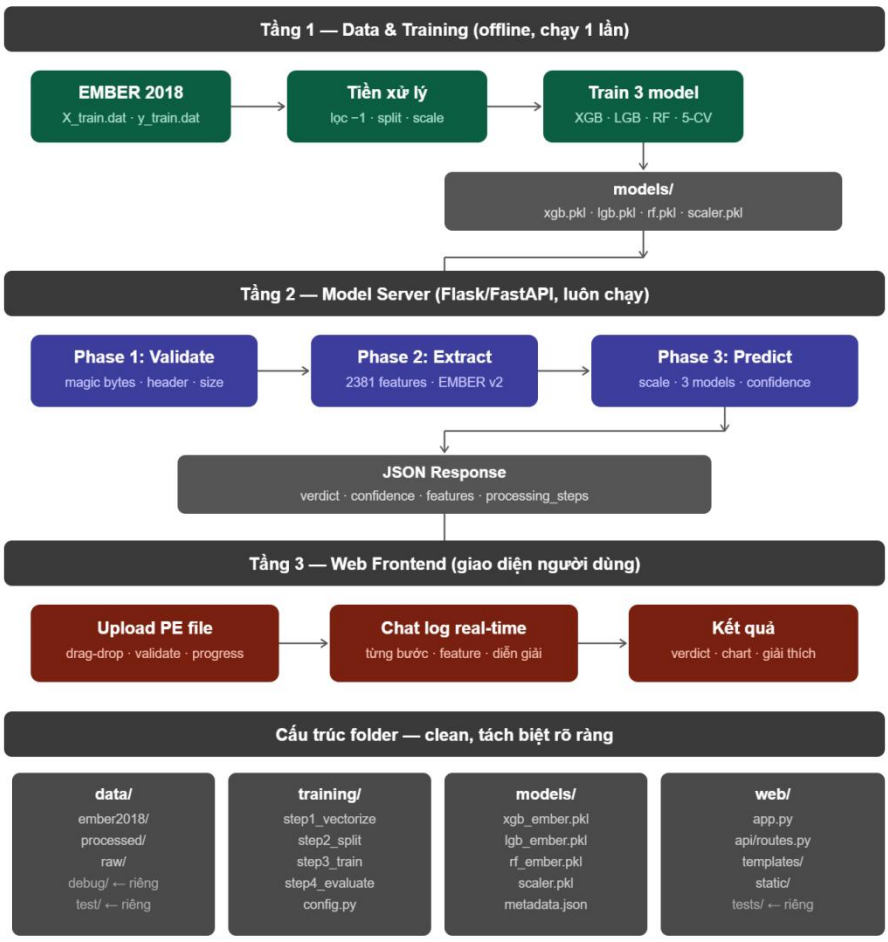
- StandardScaler được fit chỉ trên tập train (510.000 mẫu), sau đó transform riêng biệt cho train, validation và test - không fit lại trên validation hoặc test.
- 5 - Fold Stratified Cross - Validation được thực hiện thủ công theo từng fold (không dùng sklearn.cross_validate) để kiểm soát RAM với dữ liệu lớn. Mỗi fold load dữ liệu theo chỉ số, huấn luyện model, đánh giá, rồi giải phóng bộ nhớ trước khi sang fold tiếp theo.

- Holdout Stability Test: dự đoán trên 8 random split khác nhau (seed: 0, 7, 13, 42, 99, 123, 456, 2025) để xác nhận mô hình không phụ thuộc cách chia dữ liệu và không có hiện tượng overfitting.
- Robustness Test: thêm nhiễu Gaussian với $\sigma \in \{0{,}0; 0{,}1; 0{,}5; 1{,}0; 1{,}5; 2{,}0\}$ vào vector đặc trưng đã chuẩn hóa, đánh giá tính bền vững khi gặp mã độc biến thể hoặc mẫu có đặc trưng dịch chuyển.

CHƯƠNG 4: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

4.1. Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc hai thành phần độc lập: Training Pipeline (pipeline huấn luyện ngoại tuyến) và Web Application (ứng dụng web trực tuyến). Hai thành phần trao đổi qua file model đã xuất (.pkl), đảm bảo tính độc lập và khả năng cập nhật riêng lẻ.



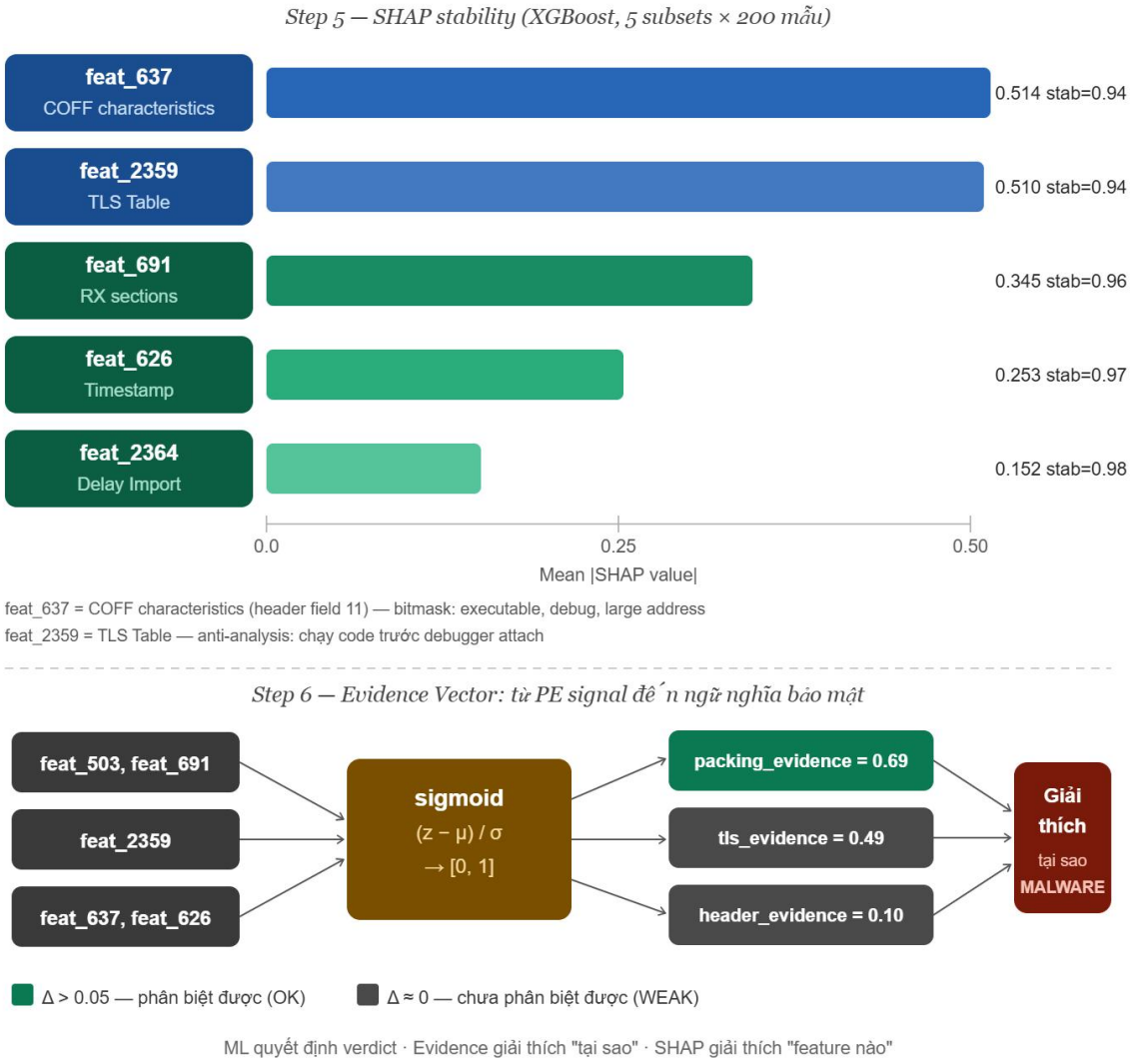
Hình 1. Kiến trúc tổng thể hệ thống phát hiện mã độc

Pipeline xử lý khi người dùng upload file PE gồm năm pha tuần tự: (1) Phase 1 kiểm tra tính hợp lệ PE - MZ header 0x4D5A, PE signature 0x50450000, kích thước

tối thiểu; (2) Phase 2 trích xuất 2.381 đặc trưng tĩnh và chuẩn hóa bằng scaler đã huấn luyện; (3) Phase 3 dự đoán bằng XGBoost với ngưỡng Youden's J và tính SHAP top features; (4) Phase 4 tính Evidence Vector năm thành phần liên tục; (5) Phase 5 nhận dạng Import Pattern theo danh mục MITRE ATT&CK.

4.2. Training Pipeline - 7 bước xử lý

Training Pipeline được chia thành 7 bước độc lập, mỗi bước lưu output ra file (.dat, .npy, .pkl) để có thể khởi động lại từ bất kỳ bước nào mà không cần chạy lại từ đầu - thiết kế hướng đến tính tái lập và dễ bảo trì.



Hình 2. Training Pipeline - sơ đồ luồng xử lý và quan hệ giữa 7 bước

Bước 1 - Vectorize (Step 1)

Chuyển đổi 6 file train JSONL (800.000 mẫu) và 1 file test JSONL (200.000 mẫu) sang định dạng numpy memmap .dat. Do EMBER PEFeatureExtractor sử dụng multiprocessing.Pool không tương thích với Python 3.13 trên Windows, quá trình vectorize được cài đặt theo vòng lặp đơn luồng. Kết quả: X_train.dat (7.266 MB) và X_test.dat (1.817 MB), mỗi file lưu ma trận float32 hình dạng (N × 2.381).

Hạn chế: Trong quá trình triển khai, đã phát sinh ràng buộc phần cứng đáng kể: ma trận đặc trưng toàn bộ tập huấn luyện có kích thước 800.000 × 2.381 × sizeof(float32) ~ 7,2 GB, trong khi máy tính nghiên cứu chỉ còn khoảng 7 - 8 GB RAM khả dụng tại thời điểm thực nghiệm. Do đó, không thể nạp toàn bộ dữ liệu vào bộ nhớ trong một lần duy nhất. Để khắc phục, toàn bộ pipeline được thiết kế lại theo cơ chế numpy.memmap - ánh xạ bộ nhớ (memory - mapped file) cho phép truy cập file trực tiếp từ đĩa theo từng slice mà không nạp toàn bộ vào RAM. Hệ điều hành chỉ nạp các trang dữ liệu thực sự được truy cập; các phần còn lại vẫn nằm trên đĩa. Nhờ đó, mức sử dụng RAM tối đa tại mỗi thời điểm chỉ tương đương một batch 50.000 mẫu (~ 450 MB), thay vì 7,2 GB nếu load toàn bộ. Cần nhấn mạnh rằng toàn bộ 800.000 mẫu vẫn được xử lý đầy đủ; đây là ràng buộc về hạ tầng phần cứng trong điều kiện nghiên cứu sinh viên, không ảnh hưởng đến tính hợp lệ của kết quả.

Bước 2 - Split & Scale (Step 2)

Lọc mẫu unlabeled (label = - 1), chia 85/15 Stratified, fit StandardScaler theo batch 50.000 mẫu để kiểm soát RAM (tối đa ~2 GB tại mỗi thời điểm). StandardScaler được fit chỉ trên tập train và lưu thành scaler.pkl. Transform riêng biệt cho từng tập không để lọt thông tin từ validation/test vào quá trình học scale.

Ưu điểm:

- StandardScaler fit chỉ trên tập train, transform riêng từng tập - > đúng chuẩn, không có data leakage
- Batch 50.000 mẫu giải quyết được bài toán RAM như bước 1

Hạn chế:

- Sau StandardScaler, nhiều feature PE có phân phối không chuẩn (max = 593.210 sau scale, không phải ~3) - vì một số feature có std ~0 (field sparse, hầu hết bằng 0), khiến z - score bị khuếch đại cực mạnh khi gặp outlier. Đây là nguyên nhân trực tiếp khiến Evidence Vector ở step 6 bị yếu: sigmoid tính trên z - score bất thường cho ra giá trị ~0.49 cho cả malware lẫn benign. Không có bước clip hay RobustScaler để xử lý outlier này.

Bước 3 - Train Models (Step 3)

Huấn luyện ba mô hình bằng 5 - Fold Stratified Cross - Validation thủ công. Mỗi fold load dữ liệu theo chỉ số, huấn luyện model, đánh giá F1 và AUC, lưu kết quả, giải phóng RAM trước khi sang fold tiếp theo. Model tốt nhất theo CV F1 trung bình được lưu thành lgb.pkl, xgb.pkl, rf.pkl.

Ưu điểm:

- 5 - Fold CV thủ công kiểm soát được RAM, holdout std = 0.00023 - > ổn định

Hạn chế:

- Random Forest huấn luyện 48,2 phút nhưng inference 36,5 ms/file - chậm hơn XGBoost 54 lần. Không có lý do chính đáng để đưa RF vào production, nhưng không có bước loại trừ tự động
- Không có early stopping cho XGBoost/LightGBM - > n_estimators cố định, có thể dư cây không cần thiết làm tăng thời gian inference

Bước 4 - Evaluate (Step 4)

Đánh giá toàn diện trên tập kiểm thử 200.000 mẫu: F1, AUC, Confusion Matrix, FPR, FNR theo ngưỡng 0,5 và ngưỡng τ^* từ Youden's J. Holdout Stability Test chạy 8 random seed. Đo tốc độ inference trung bình 1.000 lần cho từng mô hình.

Ưu điểm:

- Youden's J $\tau^* = 0.627$ - >FPR giảm từ 3.59% xuống 2.05% (- 43%) - số liệu thật
- Holdout stability 8 seed, std = 0.00023

Hạn chế:

- Step 4 output bị cắt ngắn - file step4_output.txt thực tế chỉ có phần load data, không có kết quả đầy đủ. Các con số F1/AUC trong báo cáo lấy từ CSV, không thể xác nhận log chạy thành công hoàn toàn

- Không có Cost - Sensitive Analysis: chưa định lượng chi phí tương đối giữa FP và FN - dùng Youden's J (coi FP và FN ngang nhau) có thể chưa tối ưu cho bài toán bảo mật thực tế (thường FN nguy hiểm hơn FP)

Bước 5 - SHAP Analysis (Step 5)

TreeExplainer áp dụng trên 1.000 mẫu kiểm thử cho ba mô hình. SHAP Stability Test chạy trên 5 subset ngẫu nhiên 200 mẫu mỗi subset, tính Stability Score = 1 - CV(|SHAP|) qua các subset.

Ưu điểm :

- Tất cả 10 feature top đều STABLE (0.89 - 0.97)
- feat_637 và feat_2359 nhất quán rank 1 - 2 trên cả LightGBM lẫn XGBoost
- Overlap 6 feature trong top - 20 của cả 3 model

Hạn chế:

- SHAP chỉ tính trên 1.000 mẫu (500 malware, 500 benign) - không phải toàn bộ 200.000 mẫu test. Đây là giới hạn tính toán, có thể ảnh hưởng đến tính đại diện
- CMS F1 = 0.7708 vs Full XGBoost F1 = 0.9642 - >chênh 0.1934. Điều này chứng minh 20 feature top SHAP + Logistic Regression tuyến tính là không đủ - nhưng báo cáo hiện tại trình bày chưa nêu rõ tại sao: vì Logistic Regression không bắt được non - linear interaction giữa các feature, không phải vì top - 20 feature không quan trọng
- SHAP Stability chỉ dùng 5 subset × 200 mẫu - sample size nhỏ

Bước 6 - Evidence Framework (Step 6)

Tính Evidence Vector liên tục cho tập kiểm thử, phân tích Evidence theo nhóm TP/TN/FP/FN, vẽ radar chart so sánh malware và benign. Evidence không tham gia vào phán quyết; Evidence chỉ phục vụ giải thích.

Ưu điểm:

- Packing Evidence Δ = +0.1199 - >phân biệt được malware/benign
- Kiến trúc "ML quyết định, Evidence giải thích" đúng về mặt thiết kế

Hạn chế:

- 4/5 evidence WEAK: tls_evidence Δ = 0.000, import_evidence Δ = - 0.0064, header_evidence Δ = - 0.0034, loadcfg_evidence Δ = +0.0297
- header_evidence giá trị 0.108 cho cả malware lẫn benign - gần như không có ý nghĩa phân biệt

- Ngưỡng sigmoid đặt tay, không calibrate từ data - >là nguyên nhân gốc của các $\Delta \sim 0$
- AggregatorHost.exe (Windows Telemetry service hợp lệ) bị Import Pattern Engine cho ip_score = 0.85 - >false positive thật trên file hệ thống

Bước 7 - Import Pattern (Step 7)

Xây dựng SUSPICIOUS_APIS dictionary 8 danh mục, đánh giá coverage trên 5.000 mẫu malware từ EMBER JSONL, tính Delta (malware score - benign score).

Ưu điểm thật:

- $\Delta = +0.1685$ - tốt hơn tls_evidence ($\Delta = 0$) rõ ràng
- Hoạt động trực tiếp trên import table thật (lief), hiển thị được tên API cụ thể

Hạn chế :

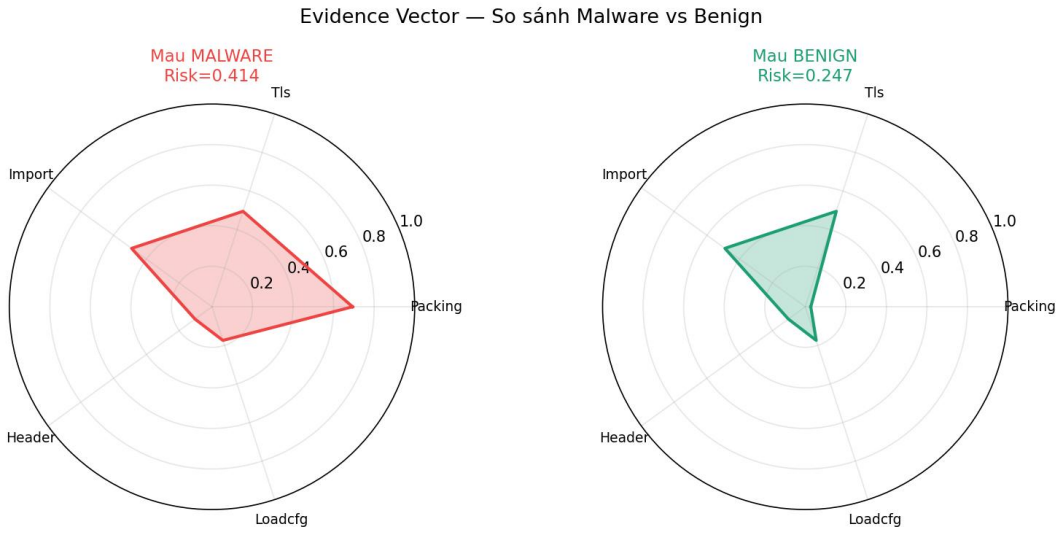
- Benign coverage 24.3% - tức gần 1/4 file lành bị flag ít nhất một API đáng ngờ. Đây là false positive rate cao ở lớp import pattern
- 54.9% malware không có API đáng ngờ nào trong dictionary - >với stager/dropper nhỏ hoặc malware không dùng các API trong list, engine hoàn toàn bỏ sót
- AggregatorHost.exe ip_score = 0.85 - Windows system service bị phán nguy hiểm, cần context từ ML score mới phân biệt được
- Engine chỉ nhìn tên API, không nhìn cách dùng API đó - CreateProcess trong calc.exe và CreateProcess trong ransomware được đối xử như nhau

Kết luận thực tế:

- Hệ thống có một lõi mạnh (bước 3+4) và một lớp giải thích thực dụng (bước 7). Bước 5 cung cấp insight học thuật có giá trị. Bước 6 là phần yếu nhất nhưng trung thực mà nói thì chính sự yếu đó cũng là kết quả nghiên cứu - nó chứng minh rằng sigmoid threshold đặt tay trên z - score sau StandardScaler không đủ để tạo ra Evidence phân biệt được ở mức population, và per - sample SHAP mới là thứ thực sự hoạt động. Đây là kết quả thực nghiệm chưa đạt kỳ vọng của một thành phần trong hệ thống.

4.3. Evidence Aggregation Framework

Evidence Aggregation Framework được thiết kế theo nguyên tắc "ML quyết định, Evidence giải thích": XGBoost là lớp phán quyết duy nhất với ngưỡng Youden's J $\tau^* = 0{,}627$; Evidence Vector đóng vai trò giải thích tại sao XGBoost đưa ra quyết định đó, không tham gia vào phán quyết.



Hình 3. Evidence Vector Radar Chart - so sánh malware (cam) và benign (xanh)

Evidence Vector gồm năm thành phần liên tục (0–1), mỗi thành phần được tính qua hàm sigmoid trên z - score đã chuẩn hóa của đặc trưng PE tương ứng. Hàm sigmoid liên tục bảo toàn gradient - entropy = 7,91 và entropy = 8,00 cho Evidence khác nhau - thay vì binarization mất thông tin:

- Packing Evidence (feat_691, Section Info): đo entropy cao bất thường trong các section - dấu hiệu nén/mã hóa. Δ = +0,120 (malware cao hơn benign một cách nhất quán).
- TLS Evidence (feat_2359, Data Directories): đo sự hiện diện TLS Table. TLS callback là kỹ thuật anti - analysis cho phép mã độc chạy code trước debugger attach.
- Import Evidence (feat_2364, Data Directories): đo Delay Import Descriptor - kỹ thuật ẩn API calls khỏi phân tích tĩnh thông thường.
- Header Evidence (feat_637, Header File Info): đo tổ hợp COFF Characteristics bất thường không phổ biến trong phần mềm hợp lệ.
- LoadCfg Evidence (feat_2360, Data Directories): đo sự vắng mặt của Load Configuration Directory - cấu trúc bảo mật thường có trong phần mềm hợp lệ được biên dịch bởi compiler hiện đại.

4.4. Import Pattern Engine

Import Pattern Engine hoạt động trực tiếp trên import table đọc từ file PE bằng LIEF, không qua FeatureHasher - do đó có thể hiển thị tên API cụ thể cho người dùng. Engine so khớp với dictionary 8 danh mục theo MITRE ATT&CK:

Danh mục	Weight	MITRE	API mẫu
Process Injection	0,95	T1055	VirtualAllocEx, WriteProcessMemory, CreateRemoteThread
Keylogger	0,90	T1056.001	SetWindowsHookEx, GetAsyncKeyState, GetKeyboardState
Network Download	0,85	T1105	URLDownloadToFile, WinHttpSendRequest, InternetOpenUrl
Crypto/Ransomware	0,80	T1486	CryptEncrypt, BCryptEncrypt, CryptGenRandom
Anti - Analysis	0,75	T1497	CheckRemoteDebuggerPresent, NtQueryInformationProcess
Persistence - Service	0,75	T1543.003	CreateService, ChangeServiceConfig, StartService
Persistence - Registry	0,45	T1547.001	RegSetValueEx, RegCreateKeyEx
Process Execute	0,40	T1059	CreateProcess, ShellExecute, WinExec

Bảng 3. Import Pattern Engine - 8 danh mục và thông tin MITRE ATT&CK tương ứng

Các danh mục có weight thấp (Persistence Registry: 0,45; Process Execute: 0,40) phản ánh thực tế rằng API như RegSetValueEx hay CreateProcess cũng xuất hiện trong phần mềm hợp lệ. Weight được xác định từ phân tích EMBER JSONL thực tế, không đặt tay tùy tiện.

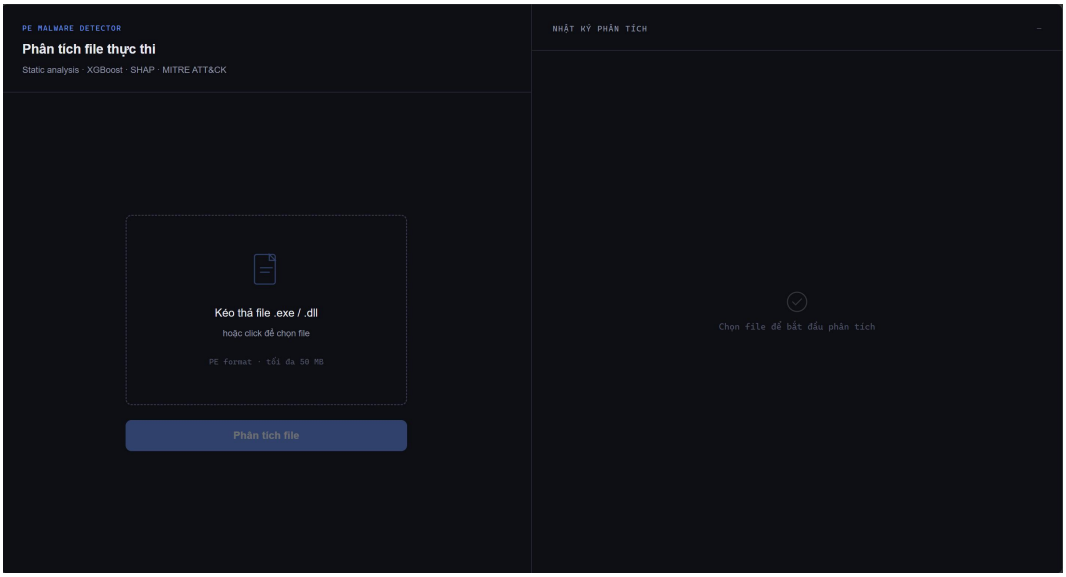
4.5. Ứng dụng web Flask

Ứng dụng web được xây dựng với Flask (backend Python) và HTML/CSS/Vanilla JavaScript (frontend), sử dụng Server - Sent Events (SSE) để truyền kết quả phân tích theo từng bước về trình duyệt theo thời gian thực. SSE được chọn thay WebSocket vì đơn hướng, không cần thư viện bổ sung, và tương thích tốt với mọi trình duyệt hiện đại.

Giao diện web có hai vùng chính: vùng upload file PE và vùng chat log hiển thị tiến trình phân tích. Sau khi pipeline hoàn thành, kết quả được hiển thị đầy đủ: verdict với thanh confidence, Evidence Vector grid, SHAP top features bar chart, Import Pattern tags gắn nhãn MITRE ATT&CK, và giải thích ngôn ngữ tự nhiên từ hàm mock_ai_interpretation- cho phép người dùng không chuyên hiểu được kết quả phân tích kỹ thuật.

a. Giao diện tổng quan

Ứng dụng web PE Malware Detector được xây dựng với giao diện tối giản trên nền tối, chia làm hai vùng chính. Vùng trái là khu vực tải lên file (.exe, .dll, tối đa 50 MB). Vùng phải là Nhật ký Phân tích — hiển thị từng bước xử lý theo thời gian thực thông qua Server-Sent Events, không cần tải lại trang. Tiêu đề trang nêu rõ ba công nghệ: Static analysis, XGBoost, SHAP và MITRE ATT&CK.



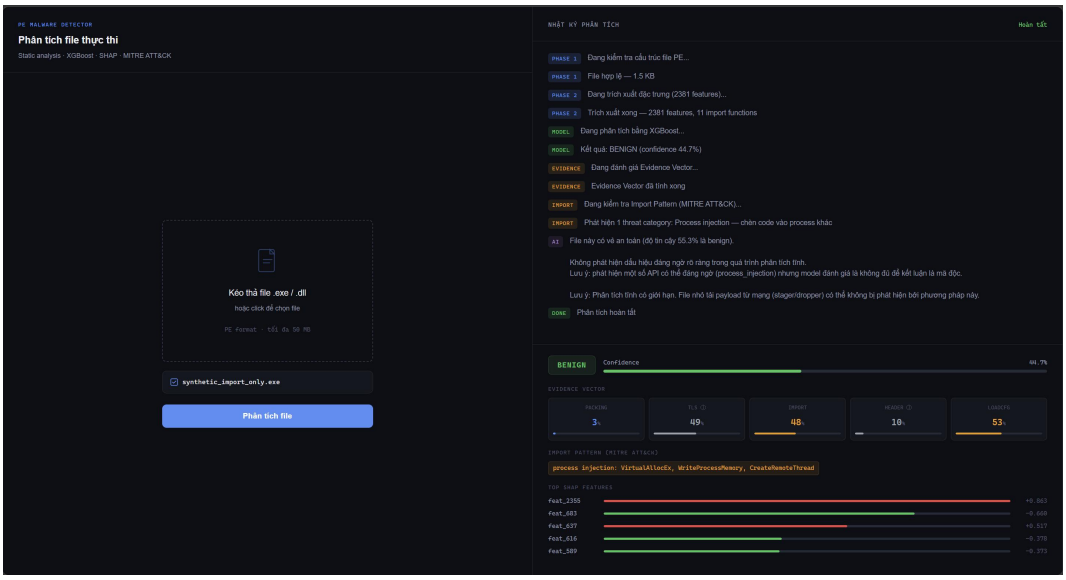
Hình 4. Giao diện chờ — vùng upload file PE và nhật ký phân tích trống

b. Nhật ký phân tích từng bước

Sau khi nhấn Phân tích file, nhật ký cập nhật từng dòng: PHASE 1 (kiểm tra cấu trúc PE), PHASE 2 (trích xuất 2.381 đặc trưng), MODEL (XGBoost phán quyết với $\tau^* = 0{,}627$), EVIDENCE (Evidence Vector 5 thành phần), IMPORT (nhận dạng API theo MITRE ATT&CK), AI (diễn giải ngôn ngữ tự nhiên), DONE (hoàn tất). Mỗi bước được đánh màu riêng để người dùng nhận ra ngay.

c. Trường hợp 1 — BENIGN (synthetic_import_only.exe, 1,5 KB)

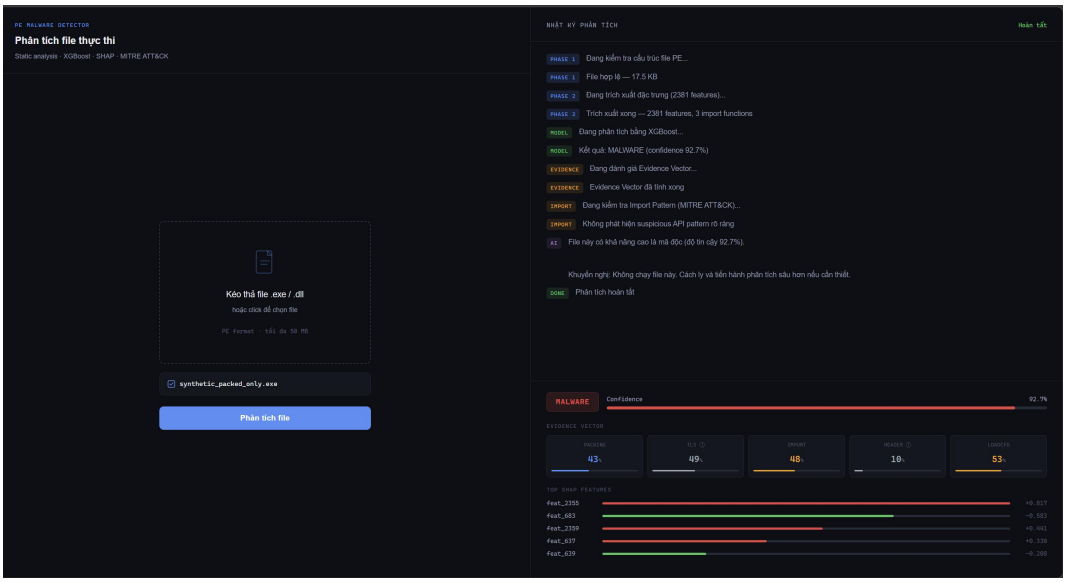
File nhỏ có import process injection nhưng mô hình phán BENIGN với độ tin cậy 44,7%. Evidence Vector: Packing = 3% (entropy thấp, không bị pack), TLS = 49%, Import = 48%, Header = 10%, LoadCfg = 53%. Import Pattern phát hiện VirtualAllocEx, WriteProcessMemory, CreateRemoteThread — phân loại Process Injection (T1055) — nhưng mô hình đánh giá không đủ bằng chứng. TOP SHAP: feat_683 và feat_589 kéo mạnh về benign (-0,560 và -0,373), feat_2355 đẩy nhẹ về malware (+0,063). AI nhận xét: không có dấu hiệu đáng ngờ rõ ràng; lưu ý phát hiện process_injection API nhưng không đủ kết luận mã độc; cảnh báo giới hạn với stager/dropper.



Hình 5. BENIGN 44,7% — phát hiện process injection API nhưng mô hình không đủ bằng chứng kết luận malware

d. Trường hợp 2 — MALWARE (synthetic_packed_only.exe, 17,5 KB)

File bị pack nặng, mô hình phán MALWARE với độ tin cậy 92,7%. Evidence Vector: Packing = 43% (tăng mạnh từ 3% ở file lành) — xác nhận entropy cao bất thường. Import Pattern không phát hiện API nguy hiểm vì file chỉ bị pack, không có import rõ ràng. TOP SHAP: feat_2355 (+0,817) và feat_2359/TLS Table (+0,441) đẩy mạnh về malware; feat_639 kéo về benign (-0,208). AI khuyến nghị: không chạy file; cách ly và phân tích sâu hơn nếu cần. Hai trường hợp xác nhận: mô hình học nhiều dạng tín hiệu; SHAP per-sample cho đóng góp khác nhau theo từng file; ba lớp giải thích bổ sung cho nhau, không thay thế nhau.



Hình 6. MALWARE 92,7% — Packing Evidence tăng lên 43%, SHAP TLS Table là tín hiệu chính (+0,441)

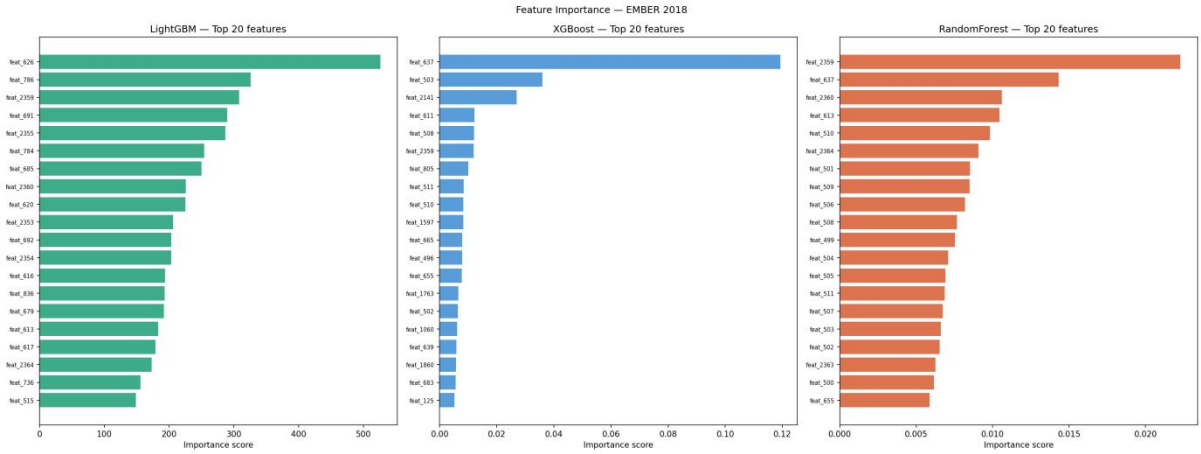
CHƯƠNG 5: KẾT QUẢ THỰC NGHIỆM VÀ THẢO LUẬN

5.1. Kết quả huấn luyện mô hình

Cả ba mô hình được huấn luyện trên 510.000 mẫu và đánh giá qua 5 - Fold Stratified Cross - Validation. Kết quả cho thấy XGBoost và LightGBM đạt hiệu suất tương đương và vượt trội Random Forest về cả F1 lẫn AUC:

Mô hình	CV F1 (TB ± ĐLC)	CV AUC	Thời gian train	Thời gian inf./file
LightGBM	0,9755 ± 0,0005	0,9972	28,3 phút	6,36 ms
XGBoost	0,9760 ± 0,0007	0,9972	65,6 phút	0,677 ms
Random Forest	0,9558 ± 0,0005	0,9933	48,2 phút	36,5 ms

Bảng 4. Kết quả Cross - Validation 5 - Fold và thời gian thực thi



Hình 7. Feature Importance - so sánh tầm quan trọng đặc trưng theo ba mô hình

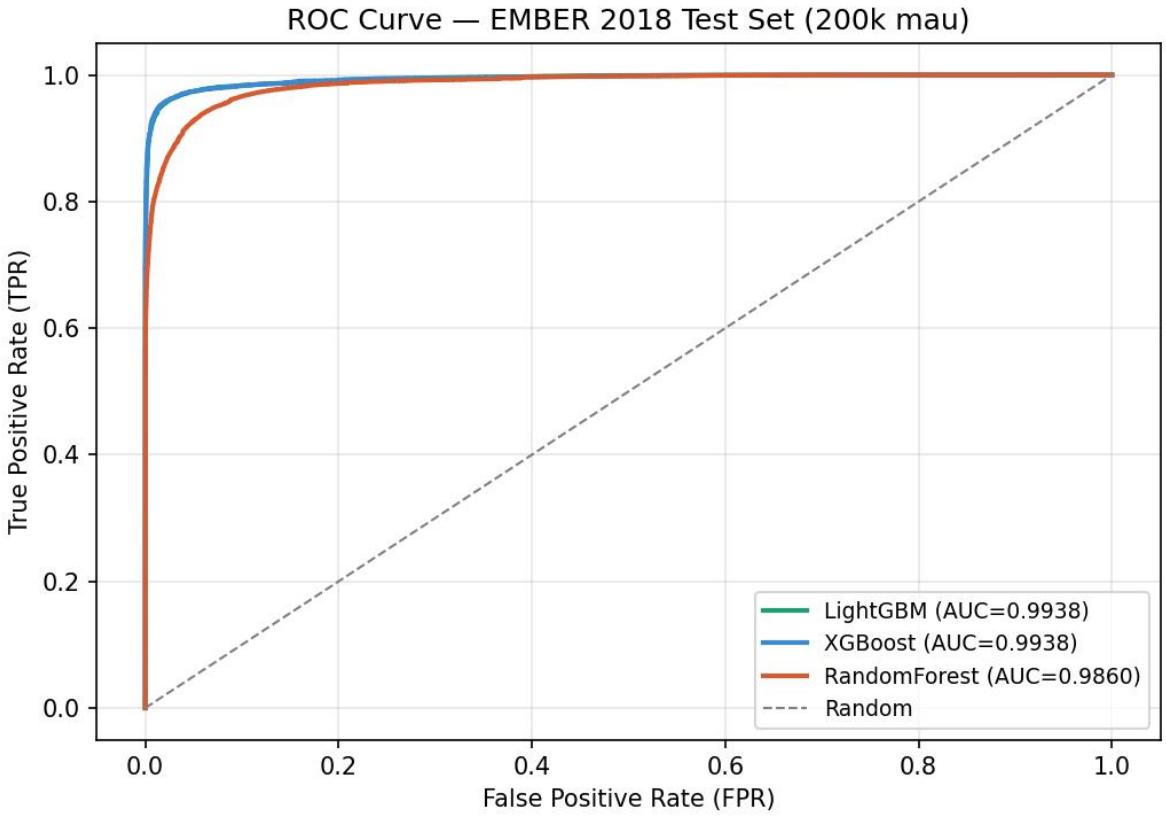
XGBoost được chọn làm mô hình production với ba lý do: (1) CV F1 tốt nhất (0,9760); (2) thời gian inference nhanh nhất (0,677 ms/file - nhanh hơn LightGBM 9,4 lần và Random Forest 54 lần trong môi trường kiểm thử); (3) tích hợp tốt nhất với TreeExplainer của SHAP cho giải thích per - sample.

5.2. Đánh giá trên tập kiểm thử độc lập

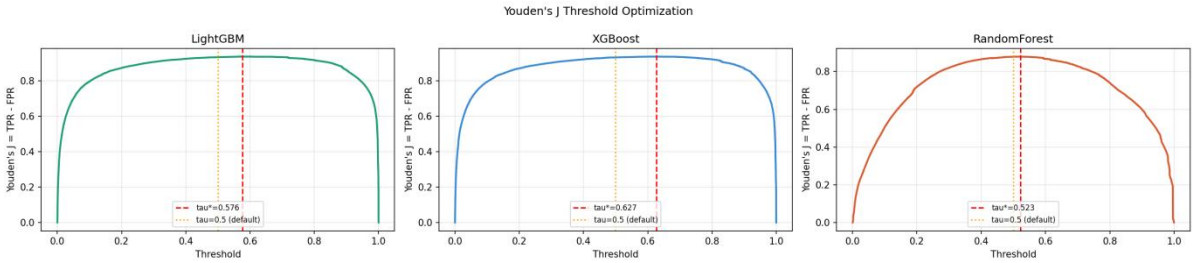
Tập kiểm thử độc lập 200.000 mẫu không được sử dụng trong bất kỳ bước huấn luyện hay tuning nào. Holdout Stability Test với 8 random seed cho độ lệch chuẩn F1 = 0,00023 đối với XGBoost, xác nhận mô hình không phụ thuộc cách chia dữ liệu và không có hiện tượng overfitting.

Mô hình	F1	AUC	FPR (τ=0,5)	FNR (τ=0,5)	τ* Youden's J	FPR (τ*)	FNR (τ*)
XGBoost	0,9682	0,9938	3,59%	3,17%	0,627	2,05%	4,24%
LightGBM	0,9675	0,9938	2,49%	3,96%	0,576	2,49%	3,96%
Random Forest	0,9395	0,9860	6,44%	5,69%	0,523	5,75%	6,30%

Bảng 5. Kết quả tập kiểm thử - ngưỡng mặc định và Youden's J tối ưu



Hình 8. ROC Curve - ba mô hình trên tập kiểm thử 200.000 mẫu



Hình 9. Threshold Analysis - Youden's J Optimization cho XGBoost

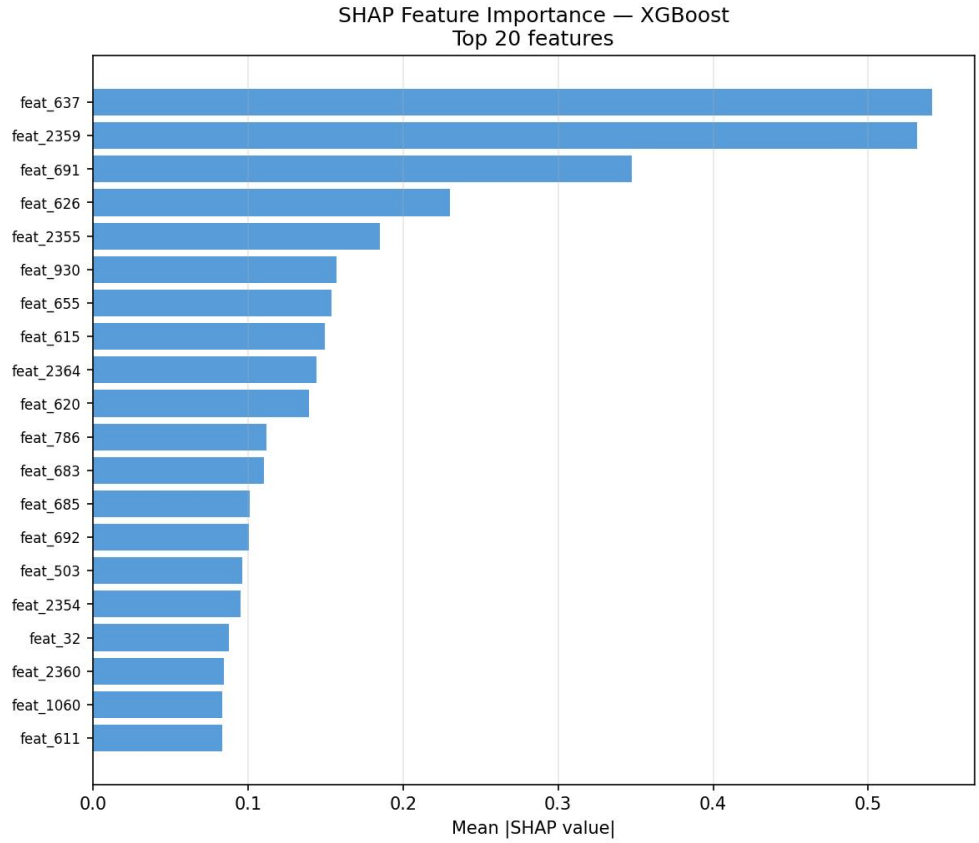
Youden's J Threshold Optimization cho XGBoost: giảm FPR từ 3,59% xuống 2,05% - tức giảm 43% số file hợp lệ bị phán nhầm là mã độc. AUC = 0,9938 vượt nhẹ kết quả nghiên cứu gốc EMBER 2018 [1] (AUC = 0,9911). Kết quả này nhất quán với tập CV (AUC = 0,9972) - chênh lệch nhỏ phản ánh domain shift tự nhiên giữa tập train (mẫu cũ hơn) và tập test (mẫu mới hơn) theo time - based split của EMBER.

5.3. Phân tích SHAP và độ ổn định đặc trưng

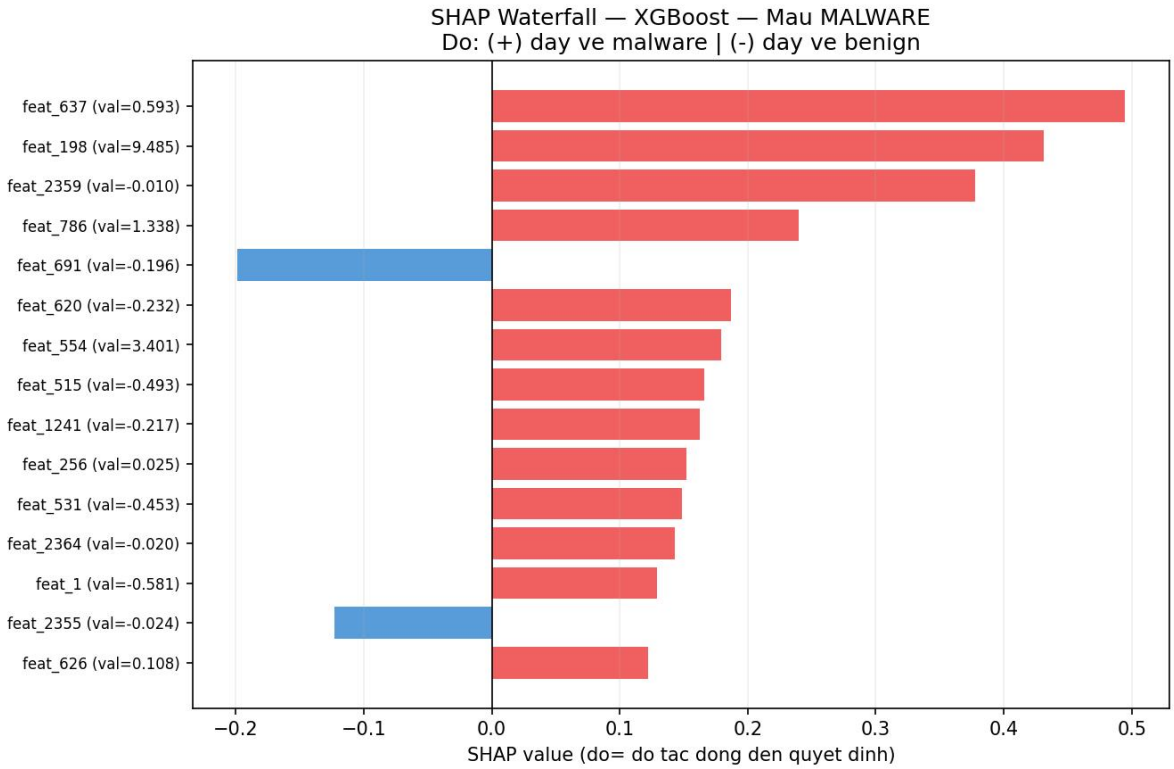
SHAP TreeExplainer được áp dụng trên 1.000 mẫu kiểm thử. Kết quả SHAP Stability Test qua 5 subset ngẫu nhiên cho thấy tất cả 10 đặc trưng quan trọng nhất đều có Stability Score > 0,89 (ngưỡng STABLE), xác nhận tính ổn định của quá trình học:

Đặc trưng	Nhóm PE	mean |SHAP|	Stability Score	Kết luận
feat_637	Header File Info (COFF Characteristics)	0,514	0,9375	STABLE
feat_2359	Data Directories (TLS Table)	0,510	0,9403	STABLE
feat_691	Section Info (entropy/RX)	0,345	0,9558	STABLE
feat_626	Header File Info (timestamp)	0,253	0,9737	STABLE
feat_2355	Data Directories	0,182	0,9544	STABLE
feat_655	Section Info	0,162	0,8945	STABLE
feat_930	Imports Info	0,157	0,9568	STABLE
feat_2364	Data Directories (Delay Import)	0,152	0,9769	STABLE
feat_615	Byte Entropy Histogram	0,148	0,9724	STABLE
feat_683	Imports Info	0,130	0,9031	STABLE

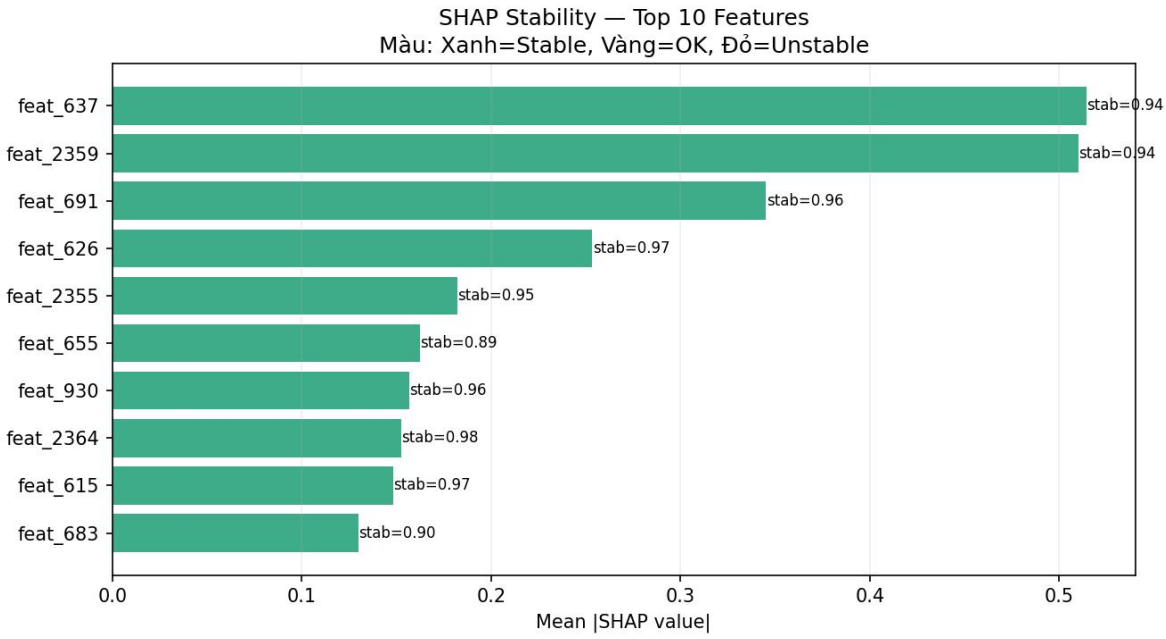
Bảng 6. SHAP Stability Score - Top 10 đặc trưng quan trọng nhất



Hình 10. SHAP Summary Plot - XGBoost, Top 10 đặc trưng (màu đỏ = giá trị cao, màu xanh = giá trị thấp)



Hình 11. SHAP Waterfall Plot - giải thích quyết định cho mẫu malware cụ thể



Hình 12. SHAP Stability Chart - Stability Score của 10 đặc trưng quan trọng nhất

Hai đặc trưng quan trọng nhất là feat_637 (COFF Characteristics, mean |SHAP| = 0,514) và feat_2359 (TLS Table, mean |SHAP| = 0,510). feat_2359 là TLS callback

- kỹ thuật anti - analysis kinh điển cho phép mã độc chạy code trước debugger attach. Việc đây là đặc trưng top - 2 xác nhận mô hình đang học đúng signal bảo mật thật, không phải noise.

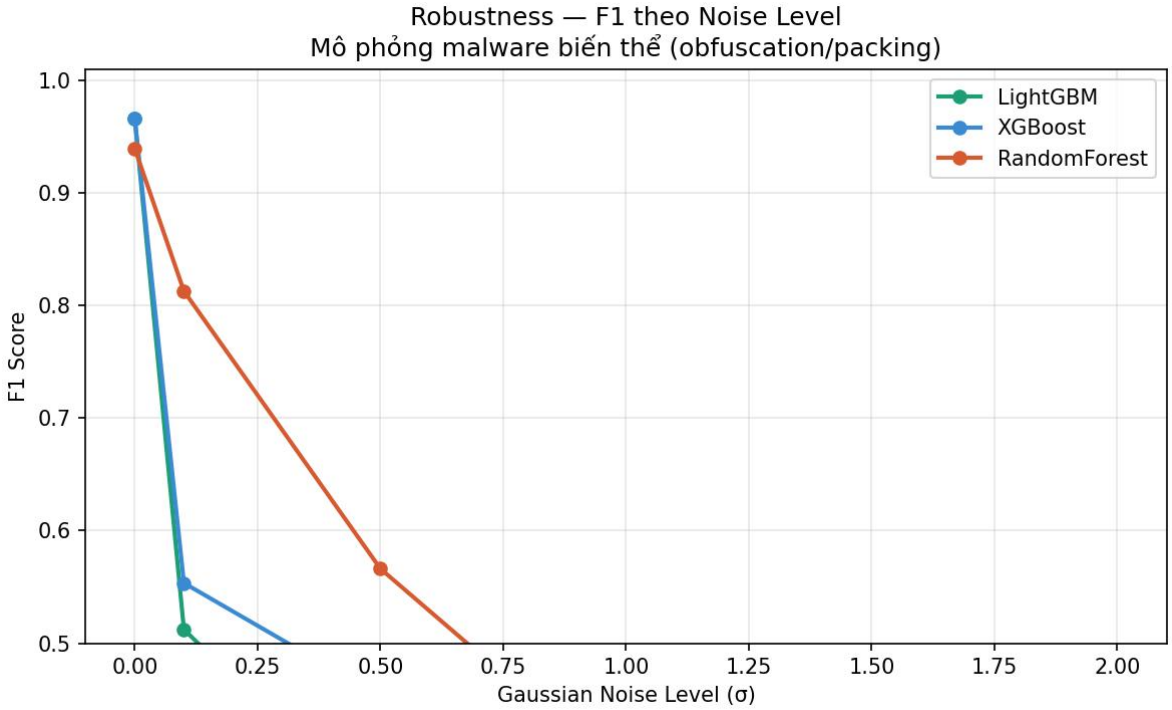
Phân tích Evidence Alignment cho thấy các đặc trưng TLS và Delay Import có mean |SHAP| cao nhất nhưng Δ population mean ~ 0. Điều này chỉ ra rằng các kỹ thuật anti - analysis này chỉ xuất hiện ở một tập con nhỏ mã độc, không phổ biến toàn dataset - lý do tại sao giải thích per - sample qua SHAP quan trọng hơn Evidence mean trong kiến trúc đã đề xuất.

5.4. Kết quả Robustness Test

Robustness Test đánh giá khả năng chống chịu nhiễu Gaussian, mô phỏng đặc tính của mã độc biến thể hoặc mẫu evasive chưa xuất hiện trong tập huấn luyện:

Mức nhiễu (σ)	XGBoost F1	LightGBM F1	Random Forest F1
0,0 (không nhiễu)	0,9663	0,9662	0,9396
0,1	0,5535	0,5122	0,8129
0,5	0,4524	0,3426	0,5662
1,0	0,4528	0,3046	0,3786
1,5	0,4702	0,3080	0,2744
2,0 (tỷ lệ giảm)	0,4856 (- 49,75%)	0,3156 (- 67,34%)	0,2229 (- 76,28%)

Bảng 7. Kết quả Robustness Test - F1 theo mức nhiễu Gaussian

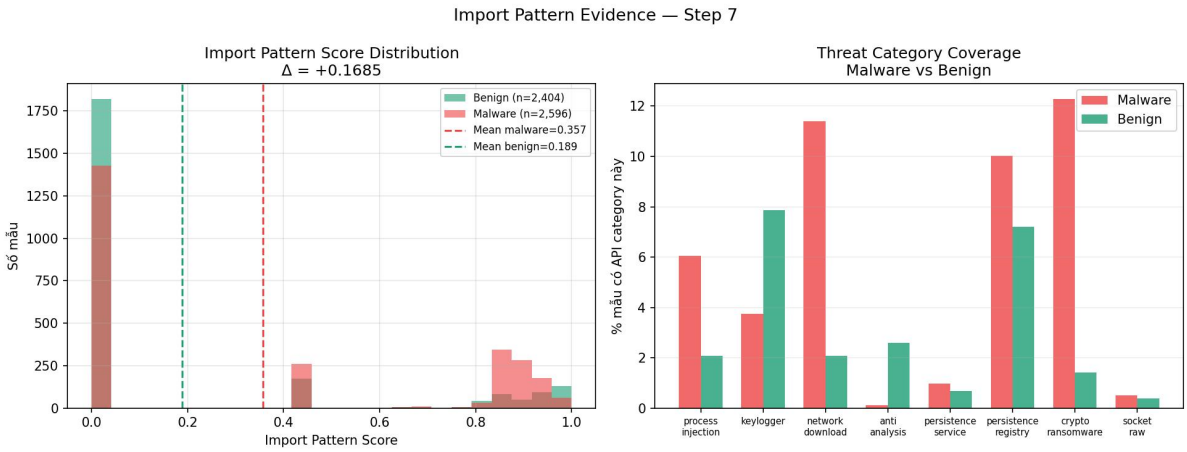


Hình 13. Robustness Curve - F1 theo mức nhiễu Gaussian σ (XGBoost giảm ít nhất)

XGBoost có tỷ lệ sụt giảm thấp nhất (49,75% tại σ = 2,0) so với LightGBM (67,34%) và Random Forest (76,28%), xác nhận XGBoost bền vững nhất với mã độc biến thể. Điểm đáng chú ý: Random Forest giảm chậm hơn ở σ = 0,1 (F1 = 0,8129) do bản chất ensemble của nó phân tán nhiễu tốt hơn trong vùng nhiễu nhỏ, nhưng sụt giảm nhanh hơn khi nhiễu lớn. Cả ba mô hình đều sụt giảm đáng kể từ σ = 0,1, phản ánh giới hạn cố hữu của static analysis trước mã độc evasive - nhất quán với kết quả challenge set của EMBER 2024 [7].

5.5. Kết quả Import Pattern Coverage

Import Pattern Engine được đánh giá trên 5.000 mẫu malware từ EMBER JSONL. Kết quả: 45,1% mẫu malware có ít nhất một API trong SUSPICIOUS_APIS list, trong khi 24,3% mẫu benign bị flag. Delta = +0,168 - cao hơn đáng kể so với TLS Evidence (Δ ~ 0) và Header Evidence (Δ = - 0,003).



Hình 14. Import Pattern Coverage - tỷ lệ phát hiện theo từng danh mục (malware vs benign)

Các danh mục có coverage cao nhất: Crypto/Ransomware (12,3%), Network Download (11,4%), Process Injection (6,0%). Điều này nhất quán với thực tế rằng ransomware và dropper chiếm tỷ lệ lớn trong mã độc hiện đại. Import Pattern Engine đặc biệt hữu ích cho trường hợp stager/dropper nhỏ nơi XGBoost confidence thấp - Engine có thể bắt được qua URLDownloadToFile và WinHttpSendRequest trong danh mục Network Download.

5.6. Hạn chế của hệ thống

Hệ thống có bốn hạn chế cần nêu rõ nhằm thể hiện tư duy nghiên cứu nghiêm túc và định hướng hướng phát triển:

- Giới hạn với stager/dropper nhỏ: File PE nhỏ tải payload từ mạng có entropy thấp và import ít, trông giống phần mềm hợp lệ từ góc nhìn phân tích tĩnh. XGBoost confidence thường thấp (0,50–0,75) cho loại này; Import Pattern Engine bắt được một phần nhưng không đảm bảo.
- Không phân tích động: Hệ thống không thực thi file trong sandbox, nên không phát hiện được fileless malware hay polymorphic malware thay đổi cấu trúc mỗi lần thực thi.
- Domain shift: EMBER 2018 được thu thập năm 2018; mã độc landscape đã thay đổi đáng kể từ đó. Mô hình cần được retrain định kỳ với dữ liệu mới để duy trì hiệu suất trên các biến thể mã độc sau năm 2018.

– *Ràng buộc RAM trong quá trình thực nghiệm:* Ma trận đặc trưng toàn bộ tập huấn luyện có kích thước ~7,2 GB (800.000 × 2.381 × float32), vượt quá dung lượng RAM khả dụng của máy nghiên cứu (7–8 GB). Hạn chế này buộc toàn bộ pipeline phải chuyển sang cơ chế batch processing và numpy.memmap thay vì xử lý in - memory. Hệ quả trực tiếp là thời gian huấn luyện kéo dài hơn đáng kể và quy trình Cross - Validation không thể song song hóa theo fold. Kết quả thực nghiệm không bị ảnh hưởng về tính chính xác, tuy nhiên trong điều kiện phần cứng có RAM ≥ 16 GB, cả ba bước vectorize, split/scale và training đều có thể tối ưu thêm đáng kể về tốc độ.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Nghiên cứu đã hoàn thành mục tiêu xây dựng hệ thống hỗ trợ phát hiện mã độc dựa trên đặc trưng tĩnh và học máy, triển khai dưới dạng ứng dụng web tương tác. Toàn bộ quy trình từ trích xuất đặc trưng đến triển khai web được cài đặt theo bảy bước pipeline độc lập, đảm bảo tính tái lập và không có data leakage.

Bốn đóng góp chính được xác nhận qua thực nghiệm nghiêm túc:

- Youden's J Threshold Optimization giảm 43% false positive rate (FPR từ 3,59% xuống 2,05%) so với ngưỡng mặc định 0,5, với căn cứ toán học rõ ràng và kết quả đo lường được trên 200.000 mẫu kiểm thử độc lập.
- Evidence Aggregation Framework (5 thành phần liên tục) kết hợp SHAP cung cấp giải thích per - sample có ngữ nghĩa bảo mật, vượt ra ngoài Feature Importance tổng thể dataset mà hầu hết nghiên cứu trước dừng lại.
- Import Pattern Engine phân loại API theo 8 danh mục MITRE ATT&CK đạt coverage 45,1% trên tập mẫu mã độc, bổ sung lớp giải thích có ngữ nghĩa tấn công cụ thể mà feature hash không thể hiển thị.
- Holdout Stability Test (std F1 = 0,00023 qua 8 random seed) và Robustness Test xác nhận tính tin cậy và khả năng tổng quát hóa của mô hình.

XGBoost đạt F1 = 0,9682 và AUC = 0,9938, vượt nhẹ kết quả của nghiên cứu gốc EMBER 2018 [1] (AUC = 0,9911), trong khi bổ sung các tầng phân tích giải thích được mà các nghiên cứu trước chưa kết hợp đầy đủ. Sử dụng hàm mock_ai_interpretation() như lớp thông dịch trung gian là điểm sáng tạo của đề tài, cho phép người dùng không chuyên hiểu được kết quả phân tích kỹ thuật.

6.2. Hướng phát triển

Dựa trên các hạn chế đã xác định, các hướng phát triển tiếp theo bao gồm:

- Tích hợp Dynamic Analysis: Kết hợp sandbox execution (Cuckoo Sandbox hoặc Any.run API) để phát hiện stager/dropper và fileless malware.
- Cập nhật dữ liệu định kỳ: Sử dụng EMBER 2024 [7] hoặc mẫu từ MalwareBazaar để retrain mô hình, đảm bảo phát hiện biến thể mã độc mới xuất hiện sau năm 2018.
- Cải thiện Evidence Calibration: Calibrate ngưỡng sigmoid của Evidence Vector dựa trên phân phối thực tế của từng đặc trưng trên validation set thay vì đặt tay.
- Mở rộng Import Pattern Engine: Bổ sung các kỹ thuật evasion mới (Heaven's Gate, Direct Syscall) và danh mục rootkit, bootkit vào SUSPICIOUS_APIS dictionary.
- Triển khai production: Chuyển từ Flask development server sang WSGI production (Gunicorn/uWSGI), thêm authentication, rate limiting và audit logging.

TÀI LIỆU THAM KHẢO

[1] Anderson, H. S., & Roth, P. (2018). EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models. arXiv preprint arXiv:1804.04637. https://doi.org/10.48550/arXiv.1804.04637

[2] Lundberg, S. M., & Lee, S. - I. (2017). A Unified Approach to Interpreting Model Predictions. In Advances in Neural Information Processing Systems (Vol. 30). Curran Associates, Inc.

[3] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). https://doi.org/10.1145/2939672.2939785

[4] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. - Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Advances in Neural Information Processing Systems (Vol. 30). Curran Associates, Inc.

[5] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

[6] Youden, W. J. (1950). Index for rating diagnostic tests. Cancer, 3(1), 32–35. https://doi.org/10.1002/1097 - 0142(1950)3:1<32::AID - CNCR2820030106>3.0.CO;2 - 3

[7] FutureComputing4AI. (2025). EMBER 2024: An Extended PE Malware Benchmark. GitHub Repository. Truy cập từ: https://github.com/FutureComputing4AI/EMBER2024

[8] MITRE Corporation. (2024). MITRE ATT&CK Enterprise Matrix (v14.1). Truy cập từ: https://attack.mitre.org/

[9] Saxe, J., & Berlin, K. (2015). Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features. In Proceedings of the 10th International Conference on Malicious and Unwanted Software (MALWARE) (pp. 11–20). https://doi.org/10.1109/MALWARE.2015.7413680

[10] Kolter, J. Z., & Maloof, M. A. (2006). Learning to Detect and Classify Malicious Executables in the Wild. Journal of Machine Learning Research, 7, 2721–2744.

PHỤ LỤC

Phụ lục A - Cấu trúc thư mục dự án

```
malware_detection_new/
├── data/
│   ├── ember2018/          # 7 file JSONL gốc + .dat sau vectorize (9 GB)
│   └── processed/          # X_train/val/test .npy, scaler.pkl (6 GB)
├── training/               # Scripts step1–step7
├── models/artifacts/       # lgb.pkl, xgb.pkl, rf.pkl, metadata.json
├── results/
│   ├── reports/            # Các file CSV và TXT output từ mỗi step
│   └── plots/              # Biểu đồ PNG (robustness, SHAP, evidence,
import)
└── web/                    # Flask app, templates, static,
requirements.txt
```

Phụ lục B - Phiên bản thư viện và công cụ

Thư viện / Công cụ	Phiên bản	Mục đích sử dụng
Python	3.13.12	Ngôn ngữ lập trình chính
XGBoost	3.2.0	Mô hình XGBoost
LightGBM	4.6.0	Mô hình LightGBM
scikit - learn	1.8.0	Metrics, StandardScaler, Cross - Validation
numpy	2.4.4	Xử lý mảng số và memmap
lief	0.17.6	Phân tích cấu trúc file PE
ember	0.1.0	Trích xuất đặc trưng PE (patched for NumPy 2.x)
shap	0.52.0	SHAP Explainability (TreeExplainer)
Flask	3.1.3	Web framework backend
pandas	3.0.3	Xử lý dữ liệu dạng bảng
joblib	1.5.3	Lưu/tải model pickle

Bảng 8. Phiên bản thư viện và công cụ sử dụng trong nghiên cứu

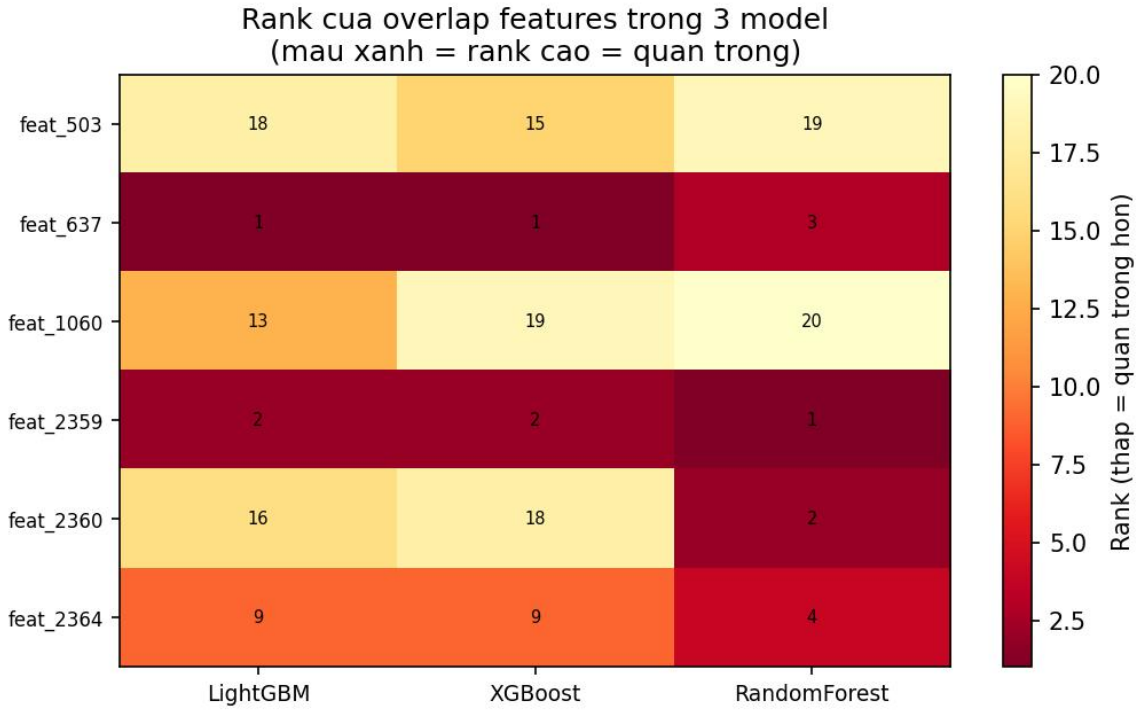
Phụ lục C - Cấu trúc JSON output của pipeline

Cấu trúc JSON response trả về từ pipeline sau khi phân tích một file PE:

```
{   "verdict"       : "MALWARE",   "confidence"    : 0.9998,   "tau_star"      :
0.627,   "evidence": {       "packing_evidence" : 0.69,       "tls_evidence"     :
0.49,       "import_evidence"  : 0.51,       "header_evidence"  : 0.72,
"loadcfg_evidence" : 0.83   },   "shap_top": [       {"feature": "feat_637",
"value": 0.514, "direction": "malware"},       {"feature": "feat_2359", "value":
```

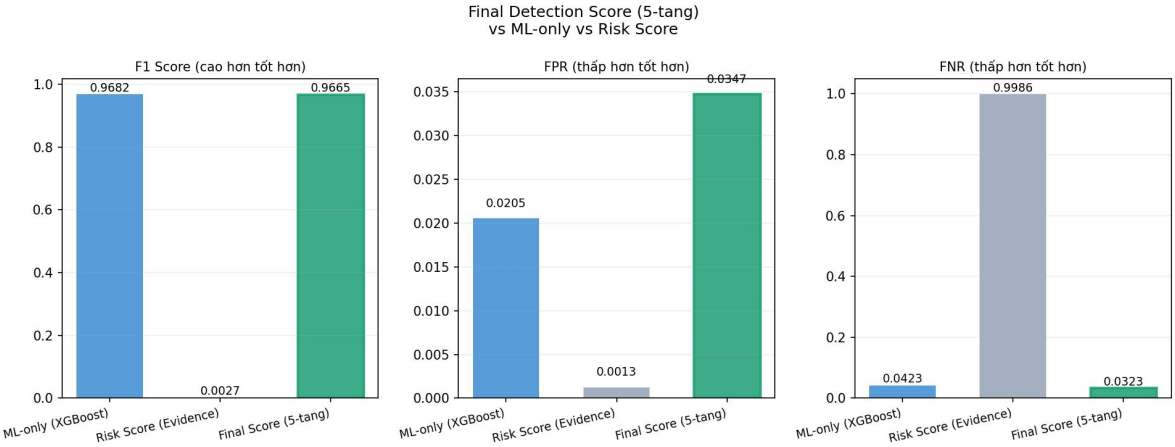
0.510, "direction": "malware"}], "import_pattern": { "score" : 0.85, "categories" : ["process_injection", "anti_analysis"], "apis_found" : ["VirtualAllocEx", "CheckRemoteDebuggerPresent"], "mitre" : ["T1055", "T1497"] } }

Phụ lục D - Kết quả thực nghiệm đầy đủ

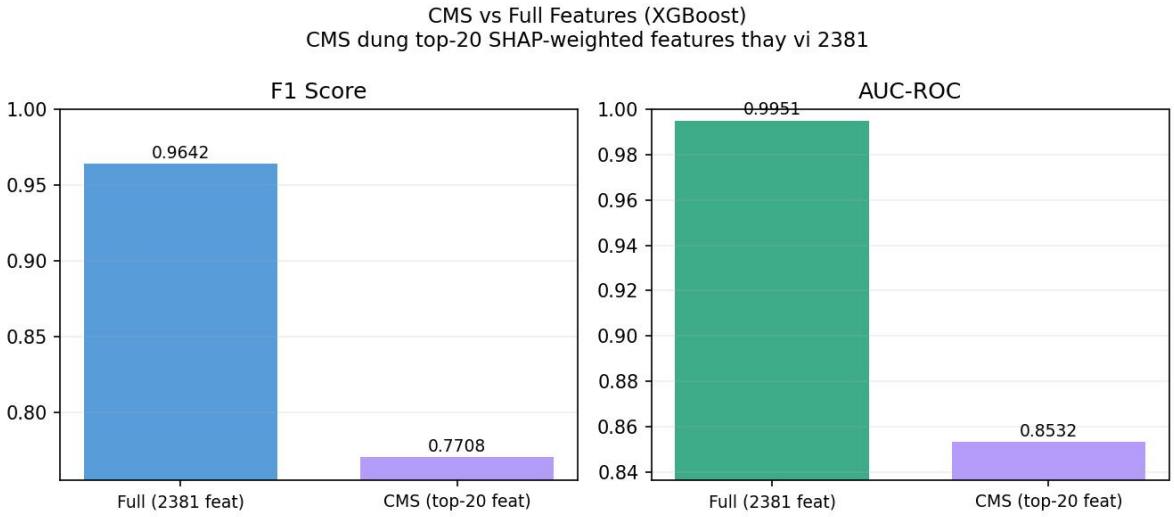


Hình D1.

Hình D 1. Overlap đặc trưng trong top - 20 SHAP giữa ba mô hình



Hình D 2. So sánh F1 / FPR / FNR giữa ML - only, Evidence - only, và Final Score



Hình D 3. So sánh CMS (Composite Malware Score) với các phương pháp đơn lẻ